General Sir John Kotelawala Defence University

Faculty of Computing



Essentials of Artificial Intelligence IT3182

Final Project Report

Smart Learning and Performance Optimization System

Group No. - 54		
Student Name	**Student ID**	**Responsibility**
Avishka Hashara	D/BCS/25/0016	Stage 1 – Neural network prediction engine, data layer and system integration
M Sadiyan	D/BIT/24/0009	Stage 2 – Mamdani fuzzy inference risk assessor
HEJM Perera	D/BIT/24/0075	Stage 3 – Genetic algorithm study hour optimiser
KSPS Amarasinghe	D/BSE/25/0023	Stage 4 – Rule based scheduler and user interface

Table of Contents

Abstract **4**
1. Introduction **5**
2. Problem Background **6**
2.1 The revision allocation problem 6
2.2 Existing approaches 7
2.3 The dataset 7
3. Problem Statement **8**
4. Objectives **9**
5. Selected AI Techniques **10**
5.1 Stage 1 – Multi layer perceptron (artificial neural network) 10
5.2 Stage 2 – Mamdani fuzzy inference system 11
5.3 Stage 3 – Genetic algorithm 11
5.4 Stage 4 – Rule based expert system (forward chaining) 11
6. Justification of AI Techniques **12**
6.1 Why a hybrid pipeline rather than a single model 12
6.2 Stage 1 – Why an MLP 12
6.3 Stage 2 – Why fuzzy logic, and why Mamdani 13
6.4 Stage 3 – Why a genetic algorithm 14
6.5 Stage 4 – Why a rule based system 14
7. System Design **15**
7.1 Architecture 15
7.2 Repository structure 15
7.3 Data flow for a single request 16
7.4 Design principles 16
7.5 User interface design 16
8. Implementation Details **17**
8.1 Stage 1 – Prediction engine (src/prediction/) 17
8.1.1 Leakage audit 17
8.1.2 Reshaping 17
8.1.3 Difficulty indicator 17
8.1.4 Synthetic confidence column (simulated data – stated explicitly) 18
8.1.5 Preprocessing and model 18
8.1.6 Production wrapper (predict.py) 19
8.1.7 PyTorch comparison (pytorch_compare.py) 19
8.2 Stage 2 – Fuzzy risk assessor (src/fuzzy/risk.py) 19
8.2.1 Universes and membership functions 19
8.2.2 Rule base 21
8.2.3 Hand worked inference example (predicted = 45, confidence = 7.5) 22
8.2.4 Label, gap and explanation 23

8.2.5 Engineering notes 23
8.3 Stage 3 – GA optimiser (src/optimizer/) 24
8.3.1 Fitness function (fitness.py) 24
8.3.2 Constraint handling (repair.py) 25
8.3.3 GA configuration (ga.py, DEAP) 26
8.3.4 Baselines (baselines.py) 27
8.4 Stage 4 – Scheduler (src/scheduler/schedule.py) 27
8.5 Integration (app/config.py) and what broke 28
Table 14. Problems found during integration and their fixes 28
9. Testing and Results 29
9.1 Stage 1 – Prediction accuracy 29
9.2 Stage 2 – Fuzzy inference acceptance tests 31
9.3 Stage 3 – GA optimisation 32
9.4 Stage 4 – Scheduler 35
9.5 End to end integration and error propagation 36
10. Limitations & Discussion 37
10.1 Limitations 37
10.1.1 Data and prediction (Stage 1) 37
10.1.2 Fuzzy assessment (Stage 2) 38
10.1.3 Optimisation (Stage 3) 38
10.1.4 Scheduling and interface (Stage 4) 38
10.2 Discussion 39
11. Ethical and Social Considerations 40
12. Conclusion 42
14. References 43
15. Individual contribution 44

Abstract

Students preparing for exams tend to split their time by how much they like a subject or how they did on earlier tests. Someone who finds a subject easy may skip it entirely. Someone anxious may keep drilling material they already know. This report describes the Smart Learning and Performance Optimization System, a four-stage pipeline built to correct that. Each stage uses the technique that fits the kind of uncertainty it has to handle.

The first stage is a multi-layer perceptron (MLP) regressor that predicts the grade a student will probably get in each subject. The second is a Mamdani fuzzy inference system. It compares that prediction with how confident the student says they feel, then outputs a study priority score from 0 to 10, a label (overconfident, aligned, or anxious), and a short explanation in plain language. The third is a real-valued genetic algorithm (GA), which divides a fixed weekly study budget across subjects to maximize expected grades while keeping each subject between a minimum and a maximum number of hours. The fourth is a rule-based scheduler that uses forward chaining to turn those allocations into a daily timetable.

We trained on data from 10,000 students. Four columns gave away the answer, so we dropped them, and the data became 40,000 student-subject rows. On 8,000 held-out rows, the MLP had a mean error of 8.18 marks. That beats our target of 10 and the student-average baseline of 10.71, but it is barely behind linear regression at 8.13. We suspect the dataset contains very little signal, so the model itself is probably not at fault. The fuzzy system passed all 12 tests, including checks for monotonicity and consistent output. The GA outperformed both equal and proportional time splits in all 30 test cases, landed close to an exhaustive search, and finished in about 0.35 seconds. Over 500 trials, the scheduler never broke a daily time limit or a session length limit. When a week doesn’t have enough hours, it cuts the shortfall instead of bending a rule. It does not add breaks yet, and it does not spread a subject evenly across days.

1. Introduction

At its core, exam revision is a question of how to spend a scarce resource. Before an exam period a student has a fixed number of hours and several subjects competing for them. Two judgements determine how those hours are spent: how well am I doing in each subject? and how much will one more hour here actually help? People are known to judge the first question poorly. Weaker performers tend to overestimate their ability (Kruger & Dunning, 1999), whereas anxious high achievers tend to underestimate theirs. Neither group has a systematic way of answering the second question.

This project develops a decision support system that answers both questions on the student's behalf and then converts those answers into a practical plan. The student supplies the number of hours currently spent on each subject and a confidence rating between 1 and 10.

From these inputs the system:

- predicts the mark the student is on course to achieve in each subject;
- challenges the student's self assessment, flagging overconfidence and anxiety and explaining each flag in plain language;
- allocates the weekly study budget across subjects so that the expected gain in marks is as large as possible; and
- schedules the allocated hours into a weekly timetable.

The system is organised as four loosely coupled modules joined by frozen interfaces (Table 1). Fixing these contracts early allowed the four team members to work in parallel and made it possible to test each stage in isolation.

Stage	Function	Receives	Returns
1	predict_scores(features_df)	DataFrame, one row per subject, unscaled features	np.ndarray (n,) of marks clipped to 0–100
2	assess_risk(predicted, confidence)	Two arrays (n,): marks 0–100, confidence 1–10	dict: priority (n,) 0–10, label list, gap (n,), explanation list
3	allocate_hours(priority, predicted, budget, min_h, max_h, seed)	Priority, marks, budget, per subject bounds	dict: hours (n,) summing to budget ±0.01, fitness, history, baselines
4	build_schedule(hours, subjects, days, daily_limits, peak_window, exam_dates)	Hours, names, days, daily cap, peak window, optional exam dates	list of session dicts (day, start, end, subject, minutes, type, reason)

Table 1. *Frozen data contract between the four stages*

2. Problem Background

2.1 The revision allocation problem

A typical undergraduate takes four to six modules at the time. Time for revision is limited by lectures, paid work, sleep and other responsibilities. So every extra hour spent on one subject means an hour less for another. Three factors make this decision harder than it seems.

1. **Students cannot see their progress:**

   The grade a student is likely to get depends on signals from their behaviour, how many hours they spend studying, how many quiz attempts they make, how often they log into the learning management system, how frequently they open course materials and how consistent their scores have been. These signals are scattered across the learning management system. No student gathers them all together. Look at the picture.

2. **Self assessment is not reliable:**

   Research in metacognition shows that confidence and actual ability often do not match exactly as we expected. The Dunning Kruger effect explains how people who perform poorly tend to rate themselves high (Kruger & Dunning 1999). At the time students with strong ability may feel anxious and over revise subjects they already understand well. Both mistakes lead to use of time but in opposite ways.

3. **The returns drop off quickly:**

   The first few hours spent on a topic usually help the most. Each additional hour adds value. Because of this, putting all time into the subject is not effective.. Since the gains are not linear, finding the best balance of time across subjects is not straightforward.

2.2 Existing approaches

Approach	**Strength**	**Shortcoming addressed by this project**
Manual planners / timetable apps	Simple and fully under the student's control	No prediction; depends entirely on biased self assessment
LMS learning analytics dashboards	Display activity data	Describe the past and do not predict, prescribe or explain
Early warning systems (e.g. at risk prediction)	Predict failure	Usually a binary flag shown to staff; no plan is given to the student
Single end to end ML recommender	Could, in principle, learn everything	Requires labelled "optimal timetables", which do not exist; not interpretable; cannot guarantee hard constraints

Table 2. *Existing approaches and the gaps this project addresses*

The comparison reveals a clear gap that no existing tool brings prediction, a calibration check, optimisation and explainable scheduling together in a single pipeline.

2.3 The dataset

The system is trained on Education_numerical.csv, which contains 10,000 students and 22 columns with no missing values. Each row records four subject scores (Maths, Science, English and History) together with 18 behavioural and contextual attributes. A leakage audit (Section 8.1) identified four columns that were derived from the target, and these were removed. Because the dataset contains no confidence column, one was simulated (Section 8.1.4). This simulation is a significant caveat and is discussed further in Sections 10 and 11.

3. Problem Statement

The system receives three inputs. a student's behavioural learning data, how confident they say they are in each of n subjects, and a fixed weekly study budget of B hours. It has to produce four things.

1. A predicted mark for each subject.
2. A judgment of whether the student's confidence matches their likely performance, with an explanation a person can follow, plus a study priority on a continuous scale.
3. A split of the B hours across subjects that maximizes the expected gain in marks, weighted by priority. Each subject must get at least its minimum hours and no more than its maximum.
4. A timetable that fits those hours into the days available and respects the scheduling rules.

It also has to do all of this quickly enough that the student isn't left waiting.

Stated formally, Stage 3 solves the following problem:

$$\max_{\mathbf{h}} F(\mathbf{h}) = \sum_{i=1}^{n} p_i \cdot (100 - \hat{y}_i) \cdot \left(1 - e^{-h_i/\tau}\right)$$

Subject to:

$$\sum_{i=1}^{n} h_i = B$$

$$h_i^{\min} \leq h_i \leq h_i^{\max}, \quad i = 1, \ldots, n$$

Here $\hat{y}$ is the prediction from Stage 1 and p_i is the priority from Stage 2. Three properties of the problem rule out a single, simple method:

- the optimiser's inputs ($\hat{y}$, p) are themselves uncertain estimates;
- one of those inputs, confidence, is a vague linguistic self report; and
- The search space is large. At a granularity of 0.5 h, dividing 40 h among 5 subjects gives C(84, 4) = 1,929,501 possible allocations. With 6 subjects the figure rises to 32,801,517, and with 8 subjects to 5,843,355,957.

4. Objectives

Every objective was given a measurable acceptance target. These targets were agreed in the project proposal before any implementation work started, so they could not be adjusted to fit the results.

	Objective	**Acceptance target**	**Achieved? (Section 9)**
1.	Predict each subject's mark from behavioural data without leakage	MAE < 10 marks on a held out test set **and** clearly better than the "student's own average" baseline	**Yes** – MAE 8.18 vs 10.71
2.	Assess confidence calibration and produce a smooth study priority with an explanation	Expected label and priority band on every case of a hand built corner case set, including the awkward middle cases	**Yes** – 12/12 tests pass
3.	Optimise the split of the hour budget	Fitness higher than both the equal and proportional splits; fast enough for interactive use	**Yes** – 30/30 seeds; ~0.35 s typical
4.	Produce a constraint respecting timetable	No hard constraint violation; all allocated hours placed	**Partially** – zero violations of implemented rules; hours dropped when the budget exceeds capacity; breaks and spacing not implemented
5.	Deliver an integrated, usable interface	End to end pipeline callable from a UI; plan readable in under one minute	**Yes (pipeline)** / not formally user tested
6.	Make the system explainable and honest	Every parameter justified in a decisions log; limitations reported	**Yes** – docs/decisions_member*.md ,this report

Table 3. *Project objectives, acceptance targets and outcomes*

5. Selected AI Techniques

The system is a hybrid intelligent system in which four AI paradigms are applied one after another (Figure 1). Each arrow between stages corresponds to one of the frozen interfaces listed in Table 1.

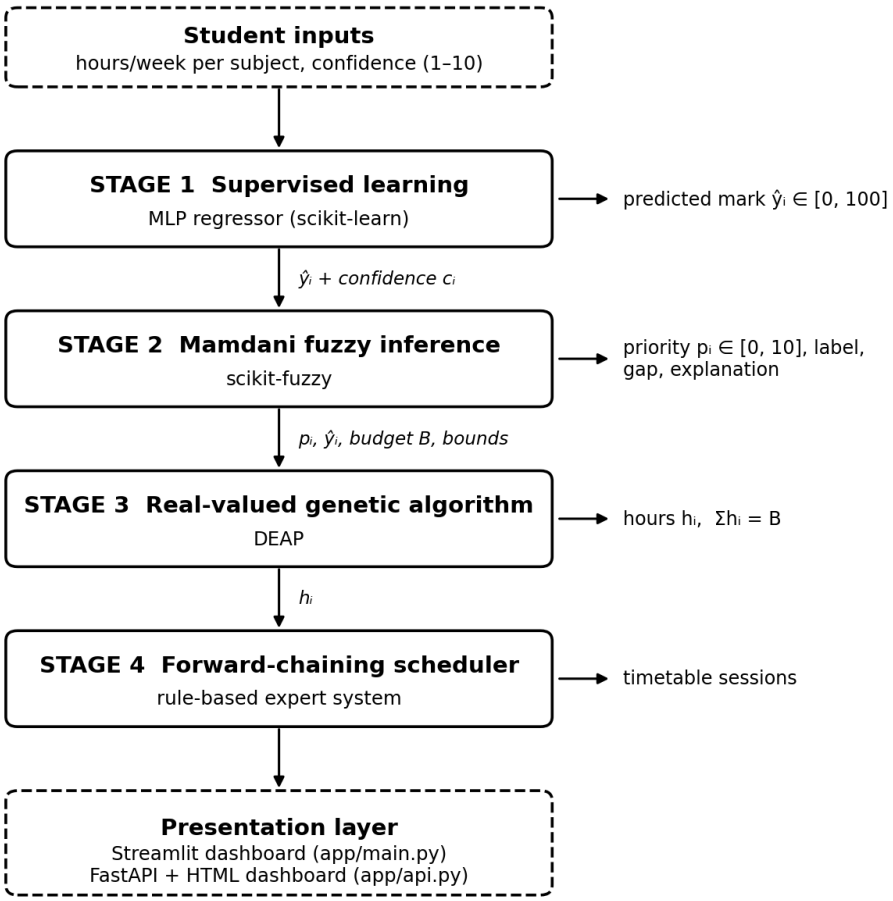


Figure 1. *The four stage pipeline. Each arrow between stages is a frozen interface (Table 1).*

5.1 Stage 1 – Multi layer perceptron (artificial neural network)

A multilayer perceptron (MLP) is a feedforward neural network. Each layer computes $a = \varphi(Wx + b)$, where W is a weight matrix learned during training, b is a bias vector, and φ is a nonlinear activation function. The weights are fitted by backpropagation, which applies the chain rule to find the gradient of the loss with respect to every weight (Rumelhart et al., 1986). Adam (Kingma & Ba, 2015) performs the update. For each parameter it tracks running estimates of the gradient's mean and variance, and it scales that parameter's step size by them. Our network has the layout 17 → 64 (ReLU) → 32 (ReLU) → 1 (linear). That comes to 3,265 trainable parameters: 1,152 in the first layer, 2,080 in the second, and 33 in the output layer.

5.2 Stage 2 – Mamdani fuzzy inference system

Fuzzy set theory (Zadeh, 1965) replaces the crisp idea of membership, where an element is either in set or not, with a degree of membership between 0 and 1. A Mamdani fuzzy inference system (Mamdani & Assilian, 1975) operates as below.

1. **Fuzzification** – crisp inputs are converted into degrees of membership.
2. **Rule evaluation** – antecedents are combined using the AND = min t norm, and each consequent set is clipped at the firing strength of its rule.
3. **Aggregation** – the clipped sets are merged using *max*.
4. **Defuzzification** – the aggregated set is reduced to a single crisp value, in this case by the centroid method.

The system has two inputs (predicted score and confidence), one output (priority) and a complete 3 × 3 rule base containing nine rules.

5.3 Stage 3 – Genetic algorithm

A genetic algorithm (Holland, 1975; Goldberg, 1989) is an example of population based meta heuristic where the concept is inspired by the process of natural selection. Candidate solutions referred to chromosomes evolve with each generation through processes of selection, crossover and mutation. This system uses a real valued encoding (one gene per subject, holding its hours), tournament selection, blend crossover (BLX α) (Eshelman & Schaffer, 1993), Gaussian mutation and elitism. Constraints are handled by a repair operator for the budget equality and by quadratic penalties for the bounds, both of which are established constraint handling strategies in evolutionary computation (Michalewicz, 1996; Coello Coello, 2002). The implementation is built on the DEAP framework (Fortin et al., 2012).

5.4 Stage 4 – Rule based expert system (forward chaining)

A rule based system stores domain knowledge as explicit IF–THEN rules, kept separate from the code that applies them. Forward chaining is a data driven style of reasoning: it begins with known facts (minutes still to be placed, empty days) and keeps firing rules until none remain applicable (Giarratano & Riley, 2005). This is the only stage of the pipeline that learns nothing, and that is a deliberate design decision.

6. Justification of AI Techniques

6.1 Why a hybrid pipeline rather than a single model

The four stages are not four alternative ways of solving the same problem. They address four distinct problems, each involving a different kind of uncertainty, as summarised in Table 4.

Stage	Kind of uncertainty	Why this technique fits
1.	Statistical: a future mark that cannot yet be observed	Supervised learning is designed to estimate an unobserved value from correlated evidence
2.	Linguistic: a vague self report ("I feel 7/10")	Fuzzy sets model categories without sharp edges, and Mamdani rules will explain their output to user
3.	Combinatorial: millions of feasible splits and a non linear objective	Population based search accepts any fitness function and any constraint handling scheme
4.	None: the rules about calendars and attention are known in advance	Explicit rules can inspected, edited and guaranteed; learning them would need data that is not existed

Table 4. *Type of uncertainty handled by each stage*

One end to end approach would need labelled instances of good schedules, which we do not have. An end to end approach would give up the interpretability that we need in Stages 2 and 4, and it would fail to guarantee that the hard scheduling constraints are satisfied. The price of the hybrid design is that errors can compound along the pipeline; this effect was measured directly and is reported in Section 9.5.

6.2 Stage 1 – Why an MLP

- **Regression rather than classification:** Marks are continuous values. For a pass/fail classifier, there is no difference between predicting 38 and 12, while for these two individuals, the required time would be different. Stage 3 requires a continuously varying prediction, such that the optimizer is faced with a smooth landscape.
- **Why a neural network:** An MLP can capture non linear interaction**s** without anyone having to construct interaction terms by hand. For example, extra study hours might be worth more to a student who also attempts quizzes. Linear regression can only represent such an effect if the interaction term is engineered explicitly.
- **Not assuming the MLP is better:** Linear regression was trained as a *control* to test whether non linearity actually helps. On this dataset it did not (Section 9.1), and this is reported as a finding in its own right.
- **Alternatives considered:** Gradient boosted trees are a strong baseline for tabular data. However, the module's emphasis on neural networks, together with the need for a smooth function of the inputs (trees produce step functions), favoured the MLP. A

PyTorch version of the same network was also built to make the training loop explicit (Section 8.1.7).

6.3 Stage 2 – Why fuzzy logic, and why Mamdani

The concrete argument. A crisp rule such as "confidence below 5 is low" would place a student who rates 4.9 and a student who rates 5.1 in opposite categories. Since priority is used as a weight inside the optimiser, the GA would then receive a discontinuous input, and a student who moved a slider by just 0.2 could see the entire timetable rearranged. With graded membership, priority becomes a continuous function of both inputs. Test C11 (Section 9.2) confirms this: moving confidence from 4.9 to 5.1 changes the priority by 0.000.

Mamdani versus Sugeno. Table 5 compares the two main fuzzy inference styles.

Criterion	**Mamdani**	**Sugeno**
Consequent	A fuzzy set ("priority is very high")	A function of the inputs (e.g. 0.8x + 2.1)
Source of rules	Human expertise	Usually fitted to data
Explanation to a student	Follows directly from the rule	A linear coefficient explains nothing
Cost	Requires defuzzification	Cheaper weighted average

Table 5. *Comparison of Mamdani and Sugeno inference*

Mamdani was chosen for two reasons. First, there is no dataset of "correct priorities" to which a Sugeno model could be fitted, so the rules must be written from expert knowledge. Second, interpretability is a functional requirement: the system has to tell the student *why* a subject is urgent.

Why centroid defuzzification: The centroid takes every fired rule into account and changes smoothly as the inputs change. Mean of maximum and similar methods ignore weakly firing rules and can jump discontinuously when the dominant rule switches, which makes them a poor source for an optimiser weight.

Why not a few if statements on the gap?: Priority is not the same as the confidence gap. A student with a predicted mark of 20 who rates their confidence at 2/10 is perfectly calibrated (gap 0), yet the subject still needs many hours. Rule R3 assigns this case a priority of 7.50 (test C2). A design based only on the gap would starve the student's weakest subject.

6.4 Stage 3 – Why a genetic algorithm

Alternative	Reason for rejection
Exhaustive enumeration	Possible for 5 subjects (1.9 M points), but the space grows to 5.8 billion for 8 subjects, and it fixes the granularity artificially
Greedy (pour hours into the best marginal subject)	Implemented as a baseline. Because the objective is separable and concave, greedy is near optimal here, so it serves as a strong benchmark rather than a straw man (Section 9.3)
Gradient methods	The quadratic bound penalties and the hard equality constraint make the surface non-smooth at the boundaries. Gradient methods would need projections or Lagrangian machinery and would not extend to arbitrary future fitness terms
Linear programming	Would solve the problem exactly **if the gain were linear**. The gain $1 - e\text{^}(-h/\tau)$ is deliberately concave, so standard LP does not apply. Convex solvers could handle this particular objective, but the GA accepts *any* fitness function the team writes, including future non separable terms (e.g. interactions between subjects)

Table 6. *Optimisation alternatives considered for Stage 3*

Each of the GA's design choices also has a specific reason:

- A real valued encoding suits the problem because hours are continuous. A binary encoding would introduce arbitrary quantisation and make arithmetic crossover meaningless.
- Tournament selection (k = 3) applies moderate selection pressure and is independent of the scale of the fitness values. Roulette wheel selection is not, which matters here because fitness grows with both the budget and the priorities.
- Blend crossover produces offspring inside the region spanned by the parents, close to the hyperplane on which the hours sum to the budget.
- Elitism guarantees that the best fitness can never decrease from one generation to the next.

6.5 Stage 4 – Why a rule based system

Scheduling constraints are determined by knowledge of how human attention and availability work, not by any patterns derived from the data set. There could be constraints on session length, session frequency, and the fact that session spacing is important for better retention (Cepeda et al., 2006; Dunlosky et al., 2013). There is no training data available for good timetables, and even if there were, the trained scheduler would be unable to put in place a hard limit and would not be able to explain the rationale behind individual placement.

7. System Design

7.1 Architecture

The system is organised into three layers.

1. **AI core (src/):**There are four independent packages (prediction, fuzzy, optimizer and scheduler), each owned by one member of the team. The only data types that move between stages are Python data types and NumPy arrays, hence there are no dependencies between packages.
2. **Orchestration (app/config.py):**The function run_pipeline(hours, confidence) creates the features dataframe, invokes the stages in sequence, computes the budget and limits, and returns all intermediate results. Both the user interfaces use only this function, and hence cannot get out of sync.
3. **Presentation:**app/main.py is the Streamlit application containing persona selector, input sliders and five result tabs. The combination of app/api.py and app/web/ contains the FastAPI JSON API (GET /api/meta, POST /api/run, POST /api/lms sync), with input data validated with Pydantic, and serves the static HTML/CSS/JS dashboard with the same five tabs (Predictions, Risk & gaps, Reallocation, Timetable, Raw data).

7.2 Repository structure

Path	Owner	Contents
src/prediction/	Avishka Hashara	prepare_data.py, baselines.py, train_mlp.py, predict.py, pytorch_compare.py
src/fuzzy/	M Sadiyan	risk.py (FIS + assess_risk), plots.py
src/optimizer/	HEJM Perera	fitness.py, repair.py, baselines.py, ga.py, allocate.py
src/scheduler/	KSPS Amarasinghe	schedule.py (build_schedule)
app/	KSPS Amarasinghe/ Avishka Hashara	config.py, main.py (Streamlit), api.py (FastAPI), web/
tests/, smoke_test.py	All	Module acceptance tests and end to end smoke test
docs/	All	Data dictionary, decisions logs, worked example, figures
models/predictor_v1.joblib	Avishka Hashara	Versioned, fitted preprocessing + MLP pipeline

Table 7. *Repository structure and ownership*

7.3 Data flow for a single request

1. The user interface sends hours[5] and confidence[5]. The API rejects any value outside hours $\in$ [1, 40] and confidence $\in$ [1, 10] with an HTTP 400 response.
2. build_features(hours) creates one row per subject. study_hours_week is taken from the user, while the remaining 12 behavioural features are set to fixed demonstration defaults that stand in for an LMS feed (Section 10).
3. Stage 1 produces $\hat{y}$ (clipped to 0–100), which Stage 2 turns into p, labels, gaps and explanations.
4. The budget is computed as B = Σ hours. hour_bounds(B, n) sets the default per subject bounds to [2 h, 15 h] and widens them where necessary so that a feasible allocation always exists (for example, B = 115 h gives a maximum of 34.5 h).
5. Stage 3 returns h, and Stage 4 builds the session list using 7 days, an 8 h daily cap and a 09:00 start time.
6. All results are returned to the interface as rounded JSON, together with the run time.

7.4 Design principles

- **Canonical subject order:**The row order of the input DataFrame is the only subject ordering in the system, and no module may sort subjects internally. Stage 3 receives no subject names, so this rule is what prevents silent misalignment.
- **Fail fast with readable errors:**Stage 1 lists any missing columns; Stage 2 checks dimensions and lengths; Stage 3 raises a specific ValueError for infeasible budgets; and Stage 4 checks that it has received one hour value per subject.
- **Reproducibility:** Seed 42 is used for the data split, the network initialisation, the synthetic confidence column and the GA.
- **Stub first integration:** Stub versions of all four functions, returning correctly shaped data, were committed early so that every member could develop against a complete pipeline from the start.

7.5 User interface design

Both dashboards follow a "read the plan in under a minute" layout, made up of three parts:

- A sidebar offering three demonstration personas (Overconfident Student, Anxious High Achiever and Uniformly Weak Student) and a simulated "Sync LMS" button;
- An input grid with hours and confidence sliders for each subject and a live budget total; and
- Five result tabs: Predictions (a bar chart of predicted marks); Risk & gaps (a classification table, a calibration bubble chart plotting confidence against predicted mark with bubble size showing priority, and plain language explanations); Reallocation (current versus optimised hours, per subject hour changes, GA versus baseline fitness

cards and the convergence curve); Timetable; and Raw data (the JSON payloads, shown for transparency).

Screenshots of the interface are stored in docs/figures/predicted_marks_tab.png and docs/figures/risk_bubble_chart.png.

8. Implementation Details

Environment: Python 3.11, pandas 2.2.1, NumPy 1.26.4, scikit learn 1.4.1, scikit fuzzy 0.4.2, DEAP 1.4.1, PyTorch 2.2.1, Streamlit 1.32.2, FastAPI ≥ 0.110, Plotly 5.20, Matplotlib 3.8.3 and joblib 1.3.2. Every version is pinned in requirements.txt, because both scikit fuzzy and DEAP have changed their APIs between releases.

8.1 Stage 1 – Prediction engine (src/prediction/)

8.1.1 Leakage audit

Each column was tested against a single question: "Would this value be known one week before the exam, and is it free of the target?" Four columns failed the test and were removed (Table 8).

Column	Reason for removal
predicted_score	An existing prediction of the target – effectively the answer
overall_avg	The mean of the four subject scores, so it contains the target
science_math_diff	The difference between two target scores
eng_hist_ratio	The ratio of two target scores

Table 8. *Columns removed by the leakage audit*

Retaining any of these columns would have produced an impressive but meaningless test score (Kaufman et al., 2012). The complete column by column audit is recorded in docs/data_dictionary.md.

8.1.2 Reshaping

Stage 2 needs one prediction for each subject, so the unit of prediction is the pair (student, subject). The four score columns were converted with `pd.melt` into a `subject` column and a target_score column, giving 10,000 students × 4 subjects = 40,000 rows.

8.1.3 Difficulty indicator

knowledge_gap (range 5–60) was converted into an ordinal difficulty indicator $\hat{y}_i \in \{1, 2, 3\}$ using equal frequency bins (`pd.qcut`, q = 3). The ordinal form is easier to explain to a user and is consistent with the agreed data contract.

8.1.4 Synthetic confidence column (simulated data – stated explicitly)

Because the dataset has no confidence rating, one was generated as follows:

$$confidence = clip(round(target_score / 10 + student_bias + \varepsilon), 1, 10)$$

$$student_bias \sim N(0, 2.0) \; - one\ draw\ per\ student\ (persistent\ bias)$$

$$\varepsilon \sim N(0, 0.5) \; - per\ row\ noise$$

This yields 11.8 % overconfident rows and 16.4 % anxious rows (|gap| > 20). Confidence is deliberately excluded from the model's features. Because it is generated from the target, using it would constitute leakage; by design, it is an *input to Stage 2*, where it is compared against the prediction.

8.1.5 Preprocessing and model

- **Features:**Thirteen numeric features (study_hours_week, quiz_attempts, consecutive_logins, material_access_freq, improvement_rate, score_consistency, time_per_question, class_size, resource_index, faculty_ratio, peak_study_time, weekend_study_ratio, difficulty) plus subject, which is one hot encoded, giving 17 inputs in total. student_id is dropped by the ColumnTransformer.
- **StandardScaler:** The features lie on very different scales (class_size ranges from 20 to 59, faculty_ratio from 0.05 to 0.20). Without scaling, gradient descent would be dominated by whichever features have the largest units.
- **OneHotEncoder(handle_unknown='ignore'):** Subject is a nominal variable, so integer codes would imply a false ordering. The ignore setting prevents the system from crashing when it meets a subject it has not seen before (see Section 10).
- **A single Pipeline:**The scaler is fitted on the training rows only, and the preprocessing steps and the model are saved together as one joblib artifact, so inference applies exactly the same transformation.

Hyperparameter	Value	Rationale
Hidden layers	(64, 32)	A funnel wide enough to learn interactions and small enough not to overfit 32 k rows
Activation	ReLU (hidden), identity (output)	Cheap and non saturating for positive inputs (Nair & Hinton, 2010); the output is linear because this is regression
Solver	Adam, lr = 0.001	Per parameter adaptive steps; robust without manual tuning
L2 penalty	α = 1e 4 (default)	Weight decay as light regularisation

Hyperparameter	Value	Rationale
Batch size	min(200, n)	Mini batch gradient descent, about 144 updates per epoch
Early stopping	10 % validation split, patience 10, tol 1e 4	Stops training before the network memorises rows (Prechelt, 1998). **Stopped at epoch 47**
max_iter	1000	Deliberately high so that early stopping, not an arbitrary cap, ends training
Split	80/20, random_state = 42	32,000 train / 8,000 test; the test set is used **once**

Table 9. *MLP hyperparameters and rationale*

8.1.6 Production wrapper (predict.py)

predict_scores() loads the model lazily and caches it in a module level variable, so it is not reloaded every time Streamlit reruns. It verifies that all 14 expected columns are present and raises a ValueError naming any that are missing. It also clips the output to [0, 100], because Stage 2's fuzzy universe spans 0–100 and an out of range value would break fuzzification.

8.1.7 PyTorch comparison (pytorch_compare.py)

The same architecture (Linear(17,64)–ReLU–Linear(64,32)–ReLU–Linear(32,1)) was re implemented in PyTorch (Paszke et al., 2019) using MSELoss, Adam (lr = 0.001) and 1,000 epochs. This makes each step of the training loop visible: zero_grad → forward → loss.backward() → optimizer.step(). The script uses CUDA when it is available. Unlike the scikit learn model, it performs full batch updates and uses no early stopping and no L2 penalty. The results of the comparison are given in Section 9.1.

8.2 Stage 2 - Fuzzy risk assessor (src/fuzzy/risk.py)

8.2.1 Universes and membership functions

Triangular functions are used for the middle terms and trapezoidal functions for the end terms. A predicted score of 5 and one of 15 are both clearly "low", and the plateau of a trapezoid expresses exactly that.

Variable (universe, step)	Term	Shape	Parameters
Predicted score (0–100, 0.5)	low	trapezoid	[0, 0, 35, 50]
	medium	triangle	**[40, 55, 75]** (tuned from [35, 55, 75])
	high	trapezoid	[60, 75, 100, 100]

Variable (universe, step)	Term	Shape	Parameters
Confidence (0–10, 0.1)	low	trapezoid	[0, 0, 3, 5]
	medium	triangle	[3, 5.5, 8]
	high	trapezoid	[6, 8, 10, 10]
Priority (0–10, 0.1)	very low / low / medium / high / very high	triangles	peaks at 0, 2.5, 5, 7.5, 10 (half width 2.5)

Table 10. *Membership functions*

Tuning decision. With the handbook's starting value of medium = [35, 55, 75], continuity test C10 failed: the priority jumped by 0.699 between predicted scores of 35 and 36. Moving the foot of the medium set to 40 preserved the low–medium overlap (40–50) and reduced the jump to 0.000. Neighbouring sets overlap by roughly 25–50 % of their width, so every point in each universe belongs to at least one set and no input can leave all rules unfired. Figures 2 to 4 show the final membership functions.

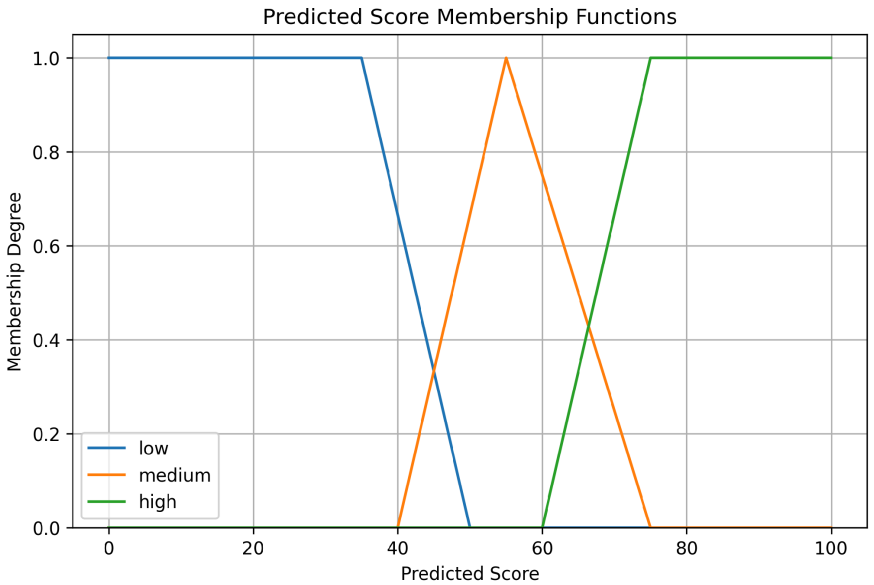


Figure 2. *Membership functions for the predicted score.*

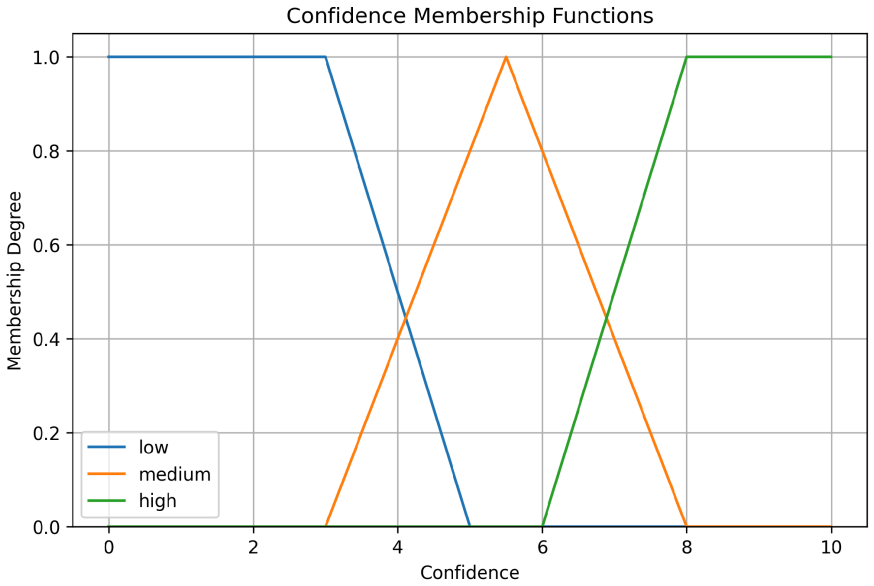


Figure 3. *Membership functions for self reported confidence.*

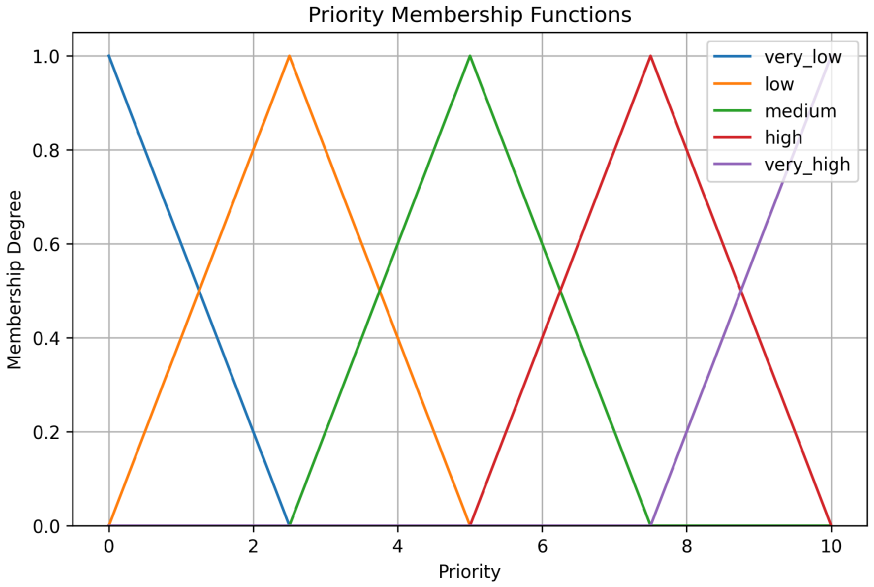


Figure 4. *Membership functions for the output priority.*

8.2.2 Rule base

The 3 × 3 grid of rules is complete by construction: every combination of input terms is covered by exactly one rule. The rule base keeps two concepts apart. Priority answers the question "how valuable is the next hour spent here?", whereas the label answers "can the student's self assessment be trusted?

	Predicted	Confidence	Priority	Reasoning
1.	low	high	very high	Failing and unaware of it – the case the system exists for
2.	low	medium	very high	Still failing and not sufficiently alarmed
3.	low	low	high	Weak and aware of it; well calibrated but still needs many hours
4.	medium	high	high	A borderline pass treated as safe; most of a typical week's marginal gain lies here
5.	medium	medium	medium	Neutral centre of the surface
6.	medium	low	medium	Borderline and worried; the hours are still useful
7.	high	high	very low	Strong and correctly confident; needs only maintenance
8.	high	medium	low	Strong and mildly unsure
9.	high	low	low	Strong but anxious; these hours are reclaimed for R1 type subjects

Table 11. *Nine rule Mamdani rule base*

8.2.3 Hand worked inference example (predicted = 45, confidence = 7.5)

To verify the implementation, one case was worked through by hand:

1. **Fuzzification:** For a predicted score of 45: μ_low = (50 − 45)/15 = **0.333**; μ_medium = (45 − 40)/15 = **0.333**; μ_high = 0. For a confidence of 7.5: μ_low = 0; μ_medium = (8 − 7.5)/2.5 = **0.200**; μ_high = (7.5 − 6)/2 = **0.750**.
2. **Rule evaluation (min):**R1 = min(0.333, 0.750) = **0.333** → very high; R2 = min(0.333, 0.200) = **0.200** → very high; R4 = min(0.333, 0.750) = **0.333** → high; R5 = min(0.333, 0.200) = **0.200** → medium. All other rules have a strength of 0.
3. **Implication and aggregation (max):** "Very high" is clipped at max(0.333, 0.200) = 0.333, "high" at 0.333 and "medium" at 0.200. The clipped shapes are then combined using *max*.
4. **Defuzzification (centroid):** The centre of gravity of the aggregated shape is **6.789**, which is exactly the value returned by assess_risk. The hand calculation therefore confirms the implementation (Figure 5).
5. **Label:** $gap = 7.5 \times 10 - 45 = +30$, which exceeds +20, so the subject is labelled overconfident.

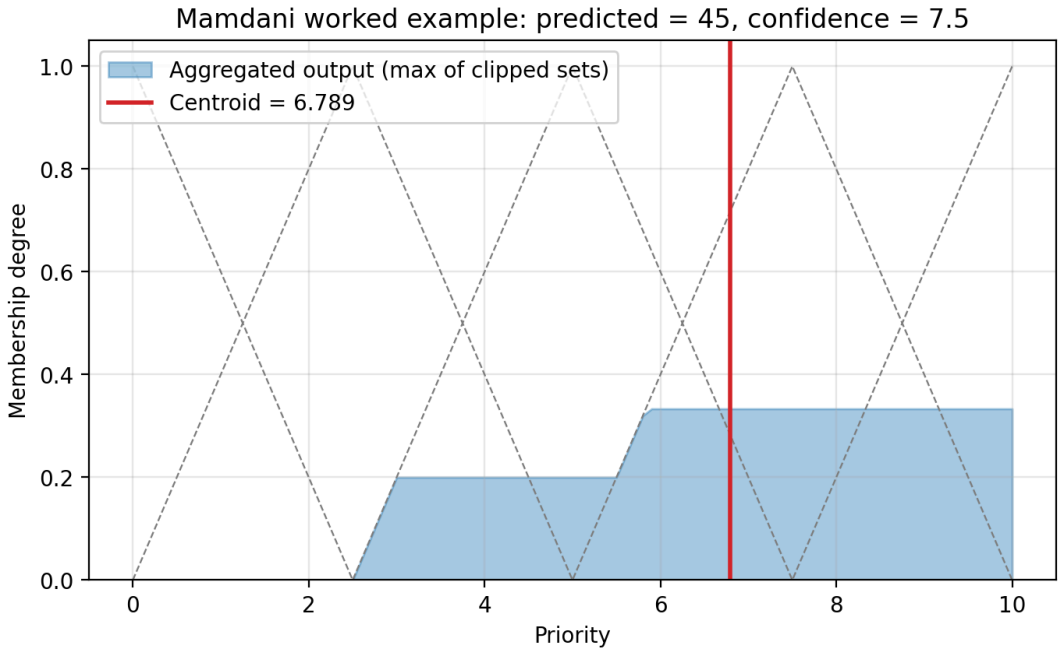


Figure 5. Mamdani worked example (predicted = 45, confidence = 7.5): aggregated output and centroid at 6.789.

8.2.4 Label, gap and explanation

The label is **not** obtained by defuzzification, because averaging a category is meaningless. Instead it is computed from a crisp gap:

$$gap = 10 \cdot confidence - predicted$$
$$gap > +20 \rightarrow overconfident, \quad gap < -20 \rightarrow anxious, \quad \text{otherwise aligned}$$

This reintroduces a crisp threshold, but only for a reported diagnostic label and not for the optimiser's weight, where crispness would cause harm. Twenty marks corresponds to about two grade boundaries, which is far too large to be a rounding effect. An alternative — taking the label from the dominant firing rule was considered and rejected, because such a label flickers near rule boundaries even when the priority barely changes. A sensitivity check on the threshold is reported in Section 9.2.

The explanation templates keep the actual numbers in the sentence. For an overconfident subject, for example: "You rated your confidence 8.0 out of 10, but your predicted score is around 32.0 … Priority is 9.17 out of 10." Separate templates cover the overconfident, anxious, aligned weak (< 50), aligned strong (≥ 75) and aligned middle cases.

8.2.5 Engineering notes

The ControlSystem is constructed only once. A fresh ControlSystemSimulation is created for each subject so that no state carries over between calls, and inputs are clamped to their universes before inference. Assessing five subjects takes approximately 25 ms.

8.3 Stage 3 – GA optimiser (src/optimizer/)

8.3.1 Fitness function (fitness.py)

The fitness function is assembled from three components and a penalty term:

$$F(h) = \Sigma_i\, p_i \cdot max(0,\ 100 - \hat{y}_i) \cdot (1 - e\text{^}(-h_i/\tau)) - \lambda \cdot P(h)$$

$$P(h) = \Sigma_i\, [\, max(0,\ L_i - h_i)^2 + max(0,\ h_i - U_i)^2\,]$$

where:

p_i : $priority$

$max(0,\ 100 - \hat{y}_i)$: $headroom$

$1 - e\text{^}(-h_i/\tau)$: $diminishing\ returns$

$\lambda \cdot P(h)$: $quadratic\ bound\ penalty$

$L_i,\ U_i$: $lower\ and\ upper\ hour\ limits\ for\ subject\ i$

Objective Function

$$F(h) = \underbrace{\sum_i p_i \cdot \max(0, 100 - \hat{y}_i) \cdot (1 - e^{-h_i/\tau})}_{} - \lambda \underbrace{\sum_i \left[(\min_i - h_i)_+^2 + (h_i - \max_i)_+^2 \right]}_{}$$

Terms in the benefit (maximize) part

p_i	Priority of subject i
$\max(0, 100 - \hat{y}_i)$	Headroom — how much the predicted score can still improve
$1 - e^{-h_i/\tau}$	Diminishing returns — additional study hours give smaller benefits over time
τ	Controls the rate of diminishing returns

Terms in the penalty part

λ	Penalty strength for violating study-hour limits
$\min_i$	Minimum allowed study hours
$\max_i$	Maximum allowed study hours
$(x)_+$	Positive-part function (max(0, x))

Other variables

h_i	Study hours allocated to subject i
$\hat{y}_i$	Predicted score for subject i

- Priority p_i (from Stage 2) expresses how much a mark gained in this subject is worth at present.
- Headroom $100 - \hat{y}_i$ is the number of marks still available. A subject predicted at 92 cannot gain 30 marks, however many hours are spent on it.
- Saturating gain $1 - e\text{^}(-h/\tau)$, with $\tau = 7$ h. Roughly 63.2 % of the available gain is captured within 7 hours (Figure 6). Because the curve is concave, a balanced split outperforms an extreme one (Table 12); with a linear gain, the problem would collapse into "give everything to the top subject".
- Penalty $\lambda = 20$. The penalty is quadratic, so small violations are cheap and large ones are prohibitive. To put its scale in context, the largest gain any one subject can contribute is $p \cdot headroom \leq 10 \times 100 = 1,000$; a 3 h violation costs 180 and a 7 h violation costs 980. In practice the repair operator (below) keeps the bounds satisfied, so the penalty acts as a secondary safeguard.

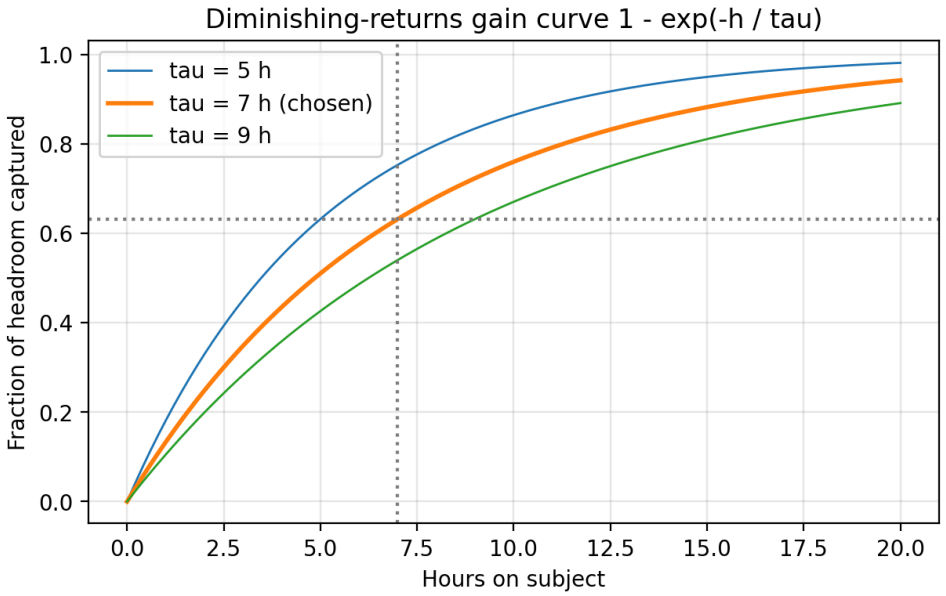


Figure 6. *Diminishing returns gain curve 1 − $exp(-\ h/\tau)$ $for\ \tau\ =\ 5,\ 7$(chosen) and 9 hours.*

$\mathbf{h_1}$	$\mathbf{h_2}$	**Fitness**
10	0	190.09
8	2	232.40
7	3	245.17
6	4	252.73
5	**5**	**255.23**

Table 12. *Concavity check*

As the model requires, the balanced split achieves the highest fitness.

8.3.2 Constraint handling (repair.py)

- **Feasibility check:** If $\Sigma min\ >\ B\ or\ \Sigma max\ <\ B$, no valid allocation can exist. In that case the function raises a specific error, such as "Sum of minimum hours (10.00h) exceeds total budget (5.00h)", rather than returning an incorrect result.
- **Bounded repair:** Scaling to the budget and then clipping to the bounds can oscillate indefinitely, because clipping changes the sum. The loop therefore alternates scale → clip at most 5 times, then distributes any remaining difference only across genes that are not pinned at a bound, in proportion to the room each has left. NaN and negative values are replaced first. Repair is applied after initialisation, after every crossover or mutation, and once more before evaluation.
- A decoder approach (storing proportions instead of hours) would remove the need for some repair altogether. Repair was chosen because it is more transparent.

8.3.3 GA configuration (ga.py, DEAP)

Table 13. GA parameters

Parameter	Value	Rationale
Encoding	Real vector, length n	Hours are continuous
Population	80	Diverse enough for 5–8 subjects (sensitivity in Table 20)
Generations	100	Best fitness plateaus within about 10 generations, leaving a wide safety margin
Selection	Tournament, k = 3	Moderate, scale invariant pressure
Crossover	cxBlend, $\alpha = 0.5$, $p = 0.8$	Offspring stay close to the feasible hyperplane
Mutation	Gaussian, $\mu = 0$, $\sigma = 1.0$ h, indpb = 0.25, $p = 0.2$	σ is about 12.5 % of B/n for B = 40, n = 5, allowing exploration without destroying good solutions
Elitism	Hall of Fame, best 2	The best so far is never lost, so the convergence curve never falls
Initial population	Random repaired vectors plus the equal, proportional and greedy baselines	Gives the search a head start and guarantees the GA never returns anything worse than the best heuristic
Seed	42 (passed in)	Reproducible results

Table 13. *GA parameters*

Before the production optimiser was finalised, an early prototype GA was developed in the genetic_algorithm/ module; this module now forwards to the production code (Section 8.5). Figure 7 shows the best and average fitness of that prototype over 40 generations. The prototype reports fitness on its own scale, so the values are not directly comparable with the production figures in Section 9.3, but the curve already shows the characteristic pattern of a rapid early rise followed by a stable plateau.

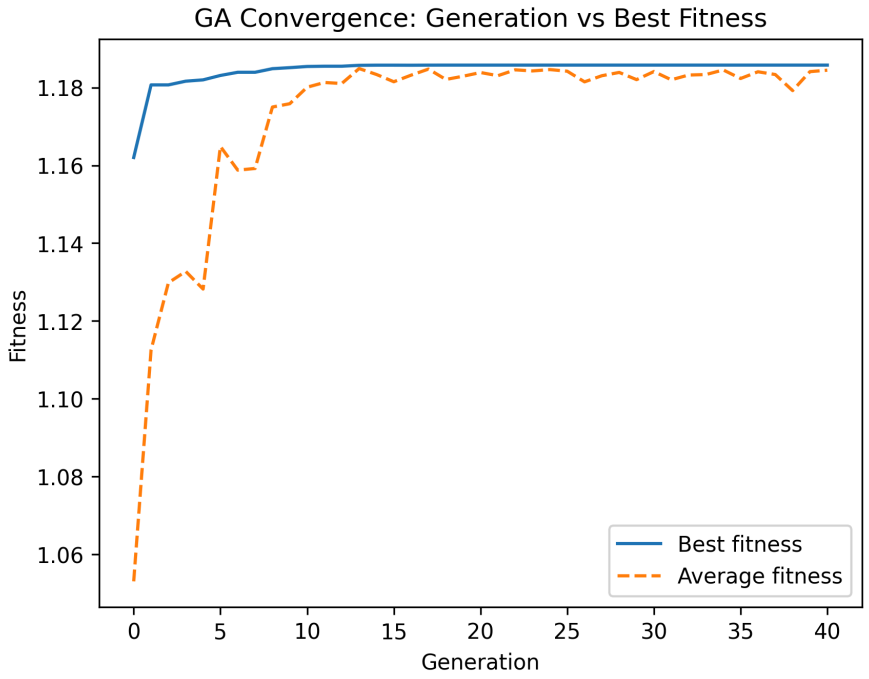


Figure 7. *Convergence of the early prototype GA: best and average fitness per generation*

8.3.4 Baselines (baselines.py)

- **Equal split:** B/n hours for every subject, followed by repair.
- **Proportional split:** hours in proportion to priority, followed by repair. This mirrors what a "worried" student would naturally do.
- **Greedy:** every subject starts at its minimum, and 0.5 h is then repeatedly added to whichever subject offers the largest marginal gain in fitness, within the bounds.

allocate_hours() returns the fitness values of all three baselines alongside the GA result. As a final safeguard it rescales the result if the sum drifts from the budget by more than 0.01 h.

8.4 Stage 4 – Scheduler (src/scheduler/schedule.py)

The scheduler is implemented as a forward chaining placement procedure, made up of a working memory, a set of rules and a defined output.

- **Working memory:** the minutes still to be placed for each subject, the current day, and the minutes already used on that day.
- **Output:** each session records day, start, end, subject, minutes, type = "study" and a reason.

The rules are applied in the following order:

1. Convert hours into whole minutes.
2. If exam dates are supplied, order the subjects by earliest exam (conflict resolution); otherwise keep the canonical order.
3. IF today's $minutes \geq daily\ limit$ THEN advance to the next day.

4. IF no days remain THEN stop.
5. Place a session of min(60, remaining minutes for this subject, remaining minutes today), starting at the peak window start time (default 09:00) plus the minutes already used today.

Input checks reject non numeric hours and mismatched lengths. An empty subject list, zero days or a non positive daily limit all return an empty schedule.

8.5 Integration (app/config.py) and what broke

Integration was not frictionless, and in keeping with the guidance not to present it as such, Table 14 records the real problems encountered and how each was resolved.

Problem found during integration	**Fix**
The app uses subjects (Databases, Networks, Statistics, Ethics) that are absent from the training data (Maths, Science, English, History). The default OneHotEncoder raised an error	Retrained with handle_unknown='ignore'. Unseen subjects map to an all zero vector. This prevents the crash, but the prediction becomes an extrapolation (Section 10)
Persona budgets (e.g. 115 h for the Anxious High Achiever) fell outside the fixed bounds n × [2, 15] = [10, 75], so check_feasibility raised an error	hour_bounds() widens the bounds based on the budget: $min = min(2, B/n)$, $max = max(15, 1.5 \cdot B/n)$
Two DEAP creator classes with the same name clashed when both the prototype GA (genetic_algorithm/) and the production GA were imported	The production classes were renamed OptimizerFitnessMax / OptimizerIndividual and guarded with hasattr; the prototype module now forwards to the production code
Streamlit reran the whole script on every widget change, reloading the model each time	Lazy, cached model loading in predict.py
The two front ends risked computing different results	Both call a single run_pipeline() in app/config.py
Out of range user inputs	Range validation in FastAPI (HTTP 400) and Pydantic list length checks; clamping in Stage 2

Table 14. *Problems found during integration and their fixes*

9. Testing and Results

Every figure reported in this section was produced by running the project code. The only values derived by hand are those in the worked example, and these agree with the code's output.

9.1 Stage 1 – Prediction accuracy

Model	MAE ↓	RMSE ↓	R^2 ↑
Global mean	8.668	10.922	0.000
Student's own average	10.705	13.442	−0.515
Linear regression	8.133	10.281	0.114
MLP (scikit learn, production)	8.177	10.342	0.103
MLP (PyTorch, full batch, 1,000 epochs)	8.884	11.225	−0.057

Table 15. *Results on the held out test set*

On the training set the MLP achieves an MAE of 8.175 and an R^2 of 0.113. On the test set, 67.7 % of predictions fall within 10 marks of the true value.

Acceptance: An MAE of 8.18 is below the target of 10, and the MLP clearly outperforms the student average baseline (10.71). Objective O1 is therefore met.

An honest interpretation: The MLP is effectively tied with linear regression and only modestly better than simply predicting the global mean. The evidence points to the data rather than the model:

1. The subject scores are uncorrelated with one another (pairwise r between −0.010 and 0.027). A student's Maths mark reveals nothing about their English mark. This also explains why the student average baseline does worse than the global mean: it averages in noise from the other subjects.
2. Permutation importance (the increase in MAE when a single feature is shuffled) is dominated by subject (1.102). The next most important features are improvement_rate (0.009), score_consistency (0.008) and material_access_freq (0.007), and the rest are close to 0. In effect, the model has mainly learned the per subject means (English 74.4, Science 69.6, History 67.4, Maths 64.5).
3. There is no overfitting. Training MAE (8.175) and test MAE (8.177) are essentially identical, and early stopping ended training at epoch 47 (Figure 8).
4. There is no non-linear structure for the MLP to exploit, so it settles close to the linear solution.

5. The PyTorch model performs worse (MAE 8.88). After 1,000 full batch steps its loss was still falling (MSE 126). It received about 1,000 parameter updates, compared with roughly 6,800 mini batch updates for the scikit learn model, which shows that the optimisation regime matters as much as the architecture.

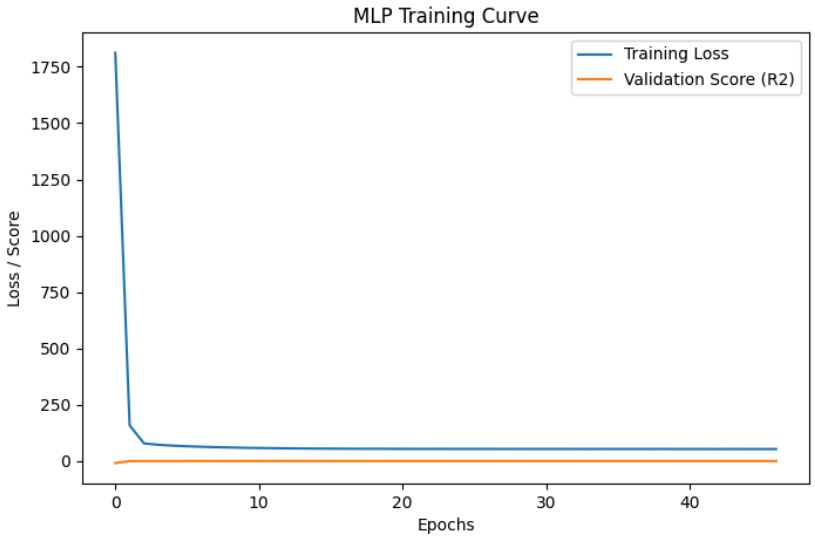


Figure 8. *MLP training curve: training loss and validation R^2 per epoch*

Error analysis of the 20 worst predictions. All 20 belong to Maths (15) or History (5), the two subjects with the widest spread (σ = 12.0 and 11.0 respectively). Their true marks lie at the extremes, either at the floor (30–37) or at the ceiling (100), while the predictions all fall within a narrow 63–71 band, giving a mean absolute error of 34.0 on these rows. The model shrinks its predictions towards the subject mean (every test prediction lies between 60.2 and 80.0), which is the expected behaviour when the signal is weak. Figure 9 shows this as a flat cloud around the diagonal. The residual plot (docs/figures/residuals.png) is centred near 0 and roughly symmetric, indicating no systematic bias.

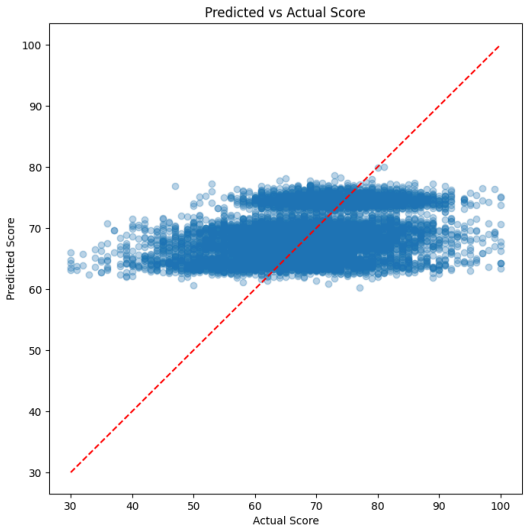


Figure 9. *Predicted versus actual score on the test set; the dashed line marks perfect prediction.*

9.2 Stage 2 – Fuzzy inference acceptance tests

Case	Predicted	Confidence	Priority	Label	Expected	Result
1.	20	9	9.167	overconfident	≥ 8, overconfident	Pass
2.	20	2	7.500	aligned	6–8.5, aligned	Pass
3.	85	9	0.833	aligned	≤ 2, aligned	Pass
4.	85	2	2.500	anxious	≤ 3.5, anxious	Pass
5.	55	5	5.000	aligned	4–6, aligned	Pass
6.	50	5	5.000	aligned	4–6, aligned (awkward middle)	Pass
7.	45	7.5	6.789	overconfident	≥ 6, overconfident (partial membership)	Pass
8.	0	10	9.167	overconfident	overconfident (extreme)	Pass
9.	100	1	2.500	anxious	anxious (extreme)	Pass
10.	35 → 36	5	9.139 → 9.139	—	Δ < 0.3 (score continuity)	Pass (Δ = 0.000)
11.	60	4.9 → 5.1	5.000 → 5.000	—	Δ < 0.2 (confidence continuity)	Pass (Δ = 0.000)
12.	0…100 step 2	5	non increasing	—	Monotonic in predicted score	Pass

Table 16. *Corner case acceptance tests (tests/test_fuzzy.py)*

All 12 tests pass, so Objective O2 is met. As is expected for centroid defuzzification, the priority spans roughly [0.83, 9.17] rather than the full [0, 10], because the centroid of an end set can never reach the edge of the universe. This behaviour was retained rather than distorting the output sets to force a wider range.

Threshold sensitivity (tests/test_fuzzy_sensitivity.py, 13 cases). Tightening the threshold from ±20 to ±15 changed 4 of the 13 labels: the four borderline cases with a gap of exactly ±20 were relabelled as overconfident or anxious. Widening it to ±25 changed none of the 13. The ±20 threshold was kept because it avoids treating borderline cases as severe miscalibration and produces the same labels as ±25 on this set.

Five subject sanity test: For predicted marks [32, 48, 58, 82, 90] and confidence ratings [8, 5, 4, 3, 9], the system returns priorities of [9.17, 5.53, 5.00, 2.50, 0.83] and labels of overconfident, aligned, aligned, anxious and aligned. When the GA is then run with B = 40 h, it gives the overconfident subject the maximum of 15.0 h and reduces the anxious but strong subject (82) to the 2 h minimum, whereas a proportional split would have given it 4.41 h. This is exactly the "reclaim hours from reassurance" behaviour the system was designed to deliver.

Control surface: Figure 10 shows the complete input–output surface. It has a high plateau of about 9.2 in the low score / high confidence corner (R1); a ridge of about 7.5 for low scores with low confidence (R3), so that honest weak students are not starved of time; a neutral shelf at 5.0 across the middle; and a floor of about 0.8 for strong, confident students (R7). Priority never increases as the predicted score increases.

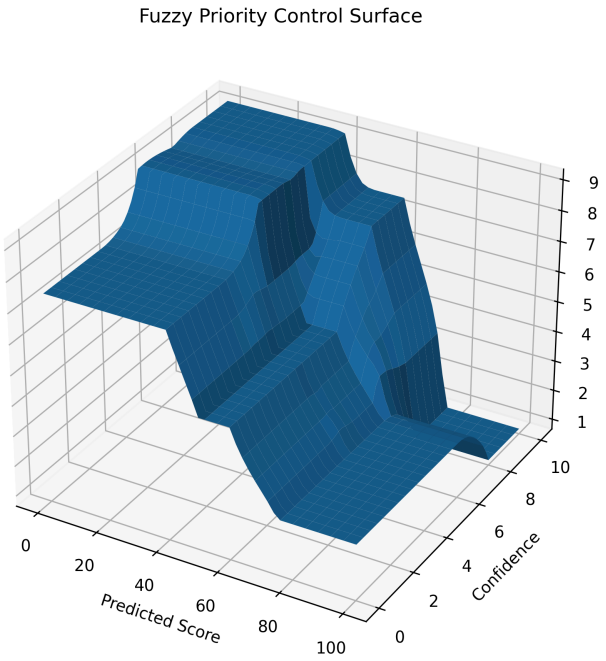


Figure 10. *Fuzzy priority control surface over predicted score and confidence.*

9.3 Stage 3 – GA optimisation

Method	**Allocation (h)**	**Fitness**
Equal	[8.0, 8.0, 8.0, 8.0, 8.0]	986.67
Proportional	[12.23, 7.48, 8.92, 3.02, 8.35]	1054.70
Greedy (0.5 h steps)	[12.0, 7.5, 9.5, 2.0, 9.0]	1063.23
GA	[12.10, 7.69, 9.42, 2.00, 8.79]	1063.32

Table 17. *Standard test case*

The GA improves on the equal split by +7.8 % and on the proportional split by +0.8 %. It moves time away from the strong subject (predicted 78.3, priority 2.1), reducing it to its minimum. Figure 11 compares the four allocations and Figure 12 shows how the GA converges.

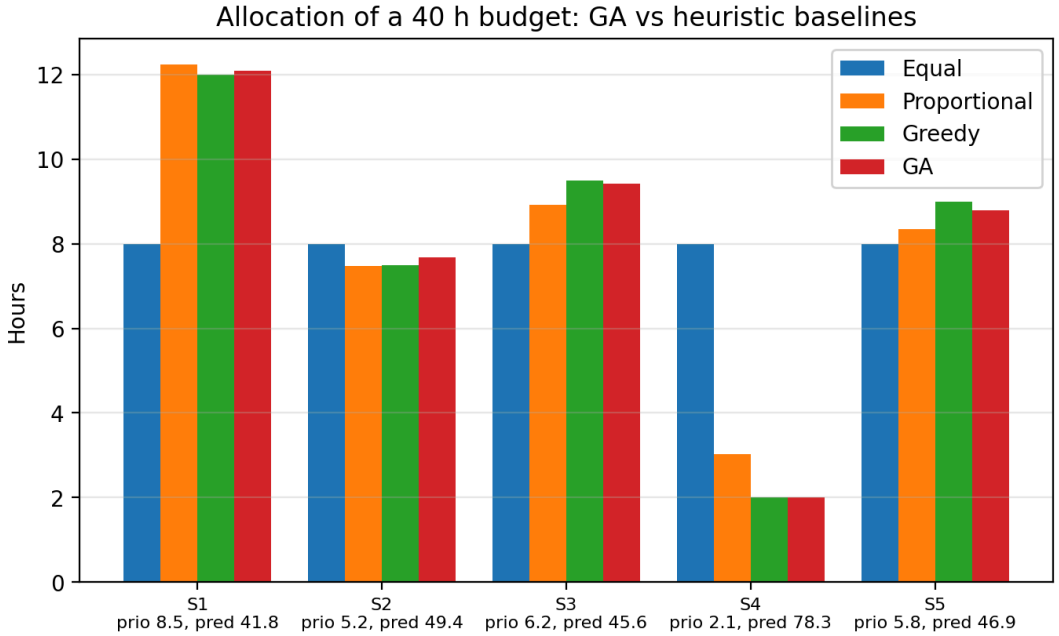


Figure 11. *Allocation of a 40 h budget: GA compared with the equal, proportional and greedy baselines.*

Statistic	**GA fitness**
Mean ± SD	1063.321 ± 0.0002
Best / worst	1063.321 / 1063.320
Beats equal	30 / 30
Beats proportional	30 / 30
≥ greedy	30 / 30

Table 18. *Robustness over 30 independent seeds (0–29)*

The variation across seeds (SD 0.0002) is negligible compared with the margin over the proportional baseline (8.6), so the GA's advantage is not a matter of luck. Objective O3 is met.

Does the result depend on the baseline seeding? It does not. All 30 seeds were rerun with a purely random initial population, with no baselines injected. The GA reached the same optimum on every seed (mean 1063.321 ± 0.0000). With seed 42 it started from 1052.8 and overtook the greedy baseline by generation 8, as Figure 12 shows. Seeding saves time, but it does not supply the answer.

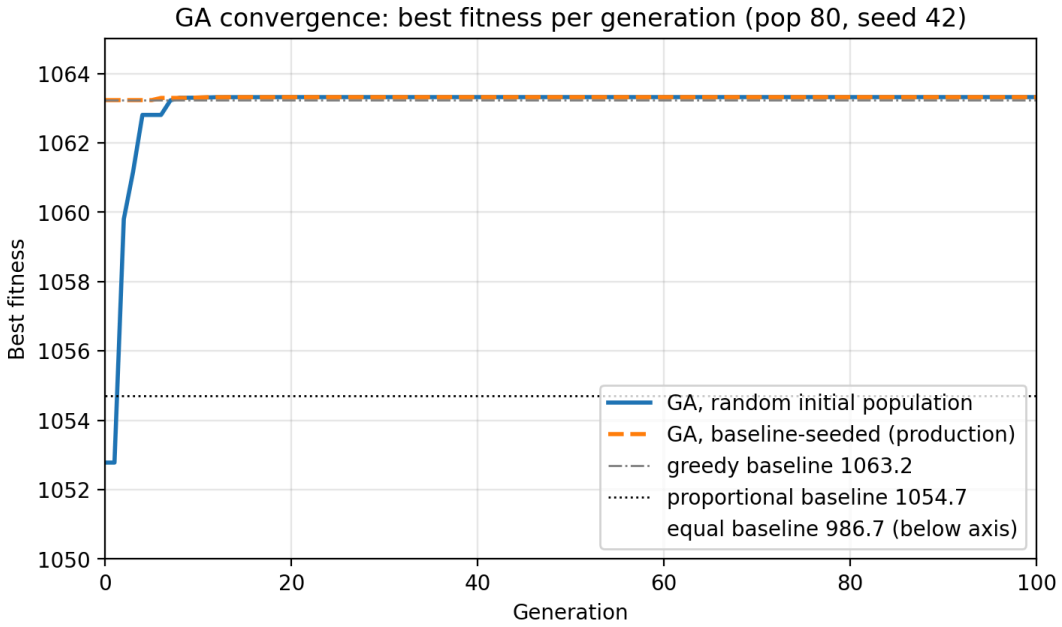


Figure 12. Production GA convergence (population 80, seed 42): random versus baseline seeded initial population.

Method	**Allocation**	**Fitness**
Exhaustive enumeration	(9.2, 4.8, 1.0)	498.436
GA	(9.21, 4.79, 1.00)	498.438

Table 19. *Brute force validation*

The GA matches the true optimum. Its value is marginally higher than the grid result because it searches on a continuous scale rather than a 0.1 h grid.

Table 20. Population size sensitivity (10 seeds each)

Population	**Mean fitness**	**SD**	**Median time**
20	1063.320	0.0008	0.09 s
50	1063.321	0.0002	0.23 s
80	1063.321	0.0002	≈ **0.35 s**
150	1063.321	0.0000	≈ 5 s (see note)

Table 20. *Population size sensitivity*

Note: on the test laptop, runs at identical settings alternated between about 0.35 s and about 2.6 3.3 s. Profiling showed a pure compute cost of under 0.9 s even with profiler overhead, so the slow runs were caused by CPU throttling rather than by the algorithm. A population of 80 delivers the same quality as 150 at a fraction of the cost, while a population of 20 is slightly less stable.

Test	Result
$\Sigma hours = budget \pm 0.01$	Pass (exactly 40.00)
All hours within [min, max]	Pass
Elitism: best fitness never decreases	Pass
Infeasible: $\Sigma min > B$ ($B = 5$)	Pass – clear ValueError
Infeasible: $\Sigma max < B$ ($B = 100$)	Pass – clear ValueError
Determinism (same seed twice)	Pass – identical
Monotone in priority (p = [9, 3], equal ŷ)	Pass – 13.85 h vs 6.15 h
All priorities identical	Pass – [5.0, 5.0, 5.0]
One priority = 0	Pass – that subject gets its minimum [2.0, 6.5, 6.5]
Budget = Σmin	Pass – [2.0, 2.0, 2.0]
Run time, 5 subjects	≈ 0.35 s typical

Table 21. *Additional Stage 3 tests*

Finding: greedy is near optimal**.** Because the objective is a sum of independent concave terms, the marginal gain greedy algorithm comes very close to the optimum; it is essentially the discrete form of equalising marginal utilities. The GA's advantage over greedy is therefore small (+0.09 in this case and +0.15 for the Uniformly Weak persona), and we report this openly. The GA remains justified because it is never worse than greedy (greedy is part of its initial population and elitism preserves it), and it will continue to work if the fitness model becomes non separable for example, with interactions between subjects or scheduling aware terms where greedy offers no guarantee.

9.4 Stage 4 - Scheduler

The scheduler was audited against its constraints using 500 randomised inputs (5 subjects, a budget drawn uniformly from 10–80 h, a random Dirichlet split, 7 days and an 8 h daily limit).

Check	Result
Any day over the 8 h limit	0 violations
Any session over 60 min	0 violations
Overlapping sessions on a day	0 violations
All minutes placed when the budget fits (≤ 56 h)	339 / 339 cases – pass
Budget larger than capacity (> 56 h)	161 / 500 cases – extra hours dropped without a warning (fail)

Check	Result
Exam date ordering	Pass – subjects with earlier exams are placed first

Table 22. *Scheduler constraint audit over 500 randomised inputs*

Inspecting a sample timetable (Databases 14 h, …) shows that subjects are placed in contiguous blocks: Databases fill Monday from 09:00 to 17:00 and continues into Tuesday. There are no breaks between sessions, and the `reason` field contains only the generic text "Rule based scheduling". The constraints that *are* implemented always hold, but the handbook's spacing, break, peak window priority and consolidation rules were not implemented. Objective O4 is therefore only partly met (Section 10).

9.5 End to end integration and error propagation

Smoke test (smoke_test.py). All three personas were run through all four stages, and each passed: the hours summed to 40.00 and a schedule was generated in every case.

Persona	Budget	Labels	GA vs equal / proportional / greedy	Pipeline time
Overconfident Student	60 h	Databases **overconfident** (gap +45.6, priority 7.80 → largest allocation 14.15 h); Maths receives the least (8.09 h, predicted 64.8)	1370.79 vs 1359.51 / 1368.22 / 1370.74	0.9 s
Anxious High Achiever	115 h	3 overconfident, 2 aligned (see Section 10)	1741.94 vs 1739.42 / 1740.62 / 1741.92	0.4 s
Uniformly Weak Student	23 h	4 anxious, 1 aligned; Maths receives the 2 h minimum	724.38 vs 696.46 / 713.26 / 724.23	2.1 s

Table 23. *Full pipeline results for the three dashboard personas*

For every persona the ordering $GA \geq greedy \geq proportional \geq equal$ holds.

Error propagation experiment: This experiment addresses the most obvious question about the architecture as a whole: "What happens if Stage 1 is wrong by ten marks?" Every Stage 1 prediction was perturbed by ±5 and ±10 marks for a profile with $\hat{y}$ = [38, 55, 62, 70, 48], confidence [8.5, 5, 6, 7, 4] and B = 40 h. The unperturbed allocation was [14.03, 7.56, 5.68, 3.38, 9.35] h.

Perturbation	Max Δpriority	Max Δhours (one subject)	Total hours moved	Labels changed
−10 marks	2.25	2.06 h	2.59 h (6.5 %)	0 / 5
−5 marks	1.39	1.08 h	1.76 h (4.4 %)	0 / 5
+5 marks	1.98	1.38 h	1.95 h (4.9 %)	0 / 5
+10 marks	1.98	1.40 h	2.71 h (6.8 %)	0 / 5

Table 24. *Sensitivity of the pipeline to Stage 1 error*

A ten mark error larger than the model's MAE of 8.2 shifts no more than about 7 % of the weekly budget, and no labels change. Because both fuzzification and the concave fitness function respond smoothly to their inputs, the plan degrades gradually rather than failing outright when a prediction is wrong.

10. Limitations & Discussion

10.1 Limitations

10.1.1 Data and prediction (Stage 1)

1. **Weak signal in the dataset:** R^2 is about 0.10, the subject scores are uncorrelated and the behavioural features carry almost no importance, all of which suggests the dataset is probably synthetic. As a consequence, study_hours_week barely moves the prediction (Maths: 65.1 at 5 h versus 66.0 at 39 h), so the system's "what if I study more?" The story is weaker than it would be with real data.
2. **The app's subjects are not in the training data:** The interface uses Databases, Networks, Statistics and Ethics, none of which the model has seen. handle_unknown='ignore' prevents a crash, but all of these subjects receive an all zero subject encoding and are predicted at about 44–46, compared with about 65 for Maths. This is why the "Anxious High Achiever" persona is labelled mostly overconfident: the persona's high confidence is being compared with extrapolated predictions rather than evidence. The proper fix is training data for those subjects, or at minimum a fallback to the mean of the closest known subject.
3. **Non hour features are fixed demonstration defaults in build_features():** A deployed system would read them from the LMS; the "Sync LMS" button is currently simulated.
4. **Row level split**: A student's other subjects can appear in the training set. The effect is small here because the subjects are uncorrelated, but a GroupShuffleSplit on student_id

would be more rigorous. In addition, the qcut bin edges were computed before the split, a minor leak of the bin boundaries.

5. **No hyperparameter search or cross validation:** The proposal planned 5 fold cross validation for model selection, but only the default architecture was evaluated.

10.1.2 Fuzzy assessment (Stage 2)

6. **Synthetic confidence:** The miscalibration rates (11.8 % and 16.4 %) describe our generator, not real students. The label thresholds were tested for sensitivity but have not been validated against genuine self reports.
7. **Hand designed membership functions:** The breakpoints come from expert judgement plus a single continuity driven adjustment, not from data. This was a deliberate choice, but it remains a limitation.

10.1.3 Optimisation (Stage 3)

8. **The fitness function is a model, not a measurement**: No study has shown that revision gain follows $1 - e^{\wedge}(-h/\tau)$, and $\tau = 7$ h was chosen by judgement. The GA is *correct with respect to its objective*; whether that objective is correct *about the world* would require a study measuring mark improvement against hours studied per topic.
9. **Greedy is almost as good under the current separable objective**, so the GA's present day advantage lies in robustness and extensibility rather than in a large fitness gain.
10. **Run time target**: A run takes about 0.35 s for 5 subjects on an unloaded CPU. The proposal's target of under 1 s for 8 subjects could not be consistently verified on the test machine because of throttling (3.3 s was measured while throttled).

10.1.4 Scheduling and interface (Stage 4)

11. **Hours are silently dropped when the budget exceeds capacity**: The 60 h Overconfident persona loses 4 h (56 h placed), and the 115 h Anxious persona loses 59 h, more than half its allocation. A capacity check that warns the user is the single most important fix.
12. **Planned rules were not implemented:** breaks between sessions, spreading subjects across days (distributed practice), giving the highest priority subject the peak window, consolidation before exams, and specific reason strings. The current output is valid but clumped (for example, 8 consecutive hours of Databases).
13. **No formal usability test**: The "plan readable in under a minute" target guided the design but was not measured with participants.
14. **Simulated budget semantics:** The budget equals the sum of the hours the student already studies. The system therefore *redistributes* existing effort and never recommends studying more or less in total.

10.2 Discussion

The project demonstrates both that a hybrid design works and where its weak points lie. The interfaces held up well: four members built their modules in parallel against frozen contracts, and the integration problems that did arise concerned the data domain (unseen subjects, budget bounds) rather than mismatched shapes or types. The interpretable stages behaved as designed. The fuzzy system's output can be traced rule by rule and was confirmed by a hand calculation, while the GA's output can be checked against brute force and simple heuristics.

The weakest link is Stage 1, and the cause lies in the data rather than the model. Every later stage depends on the predicted mark, and the error propagation experiment shows that the pipeline is *robust* to prediction error, with a 10 mark error shifting only about 7 % of the hours. The flip side is that the plan can never be *better informed* than the prediction allows. Given real, correlated LMS data, the same architecture would allow behavioural features to shape the plan in a meaningful way.

A second insight concerns the separation of priority from labels. Many naïve designs would derive urgency from the confidence gap alone. The rule base deliberately gives the honest weak student (R3) a high priority and the anxious strong student (R9) a low one. The five subject test shows the effect clearly: the GA moves hours away from an anxious student's strong subject and *towards* their overconfident weak subject — precisely the behavioural change the system is meant to support.

Future work, in order of priority:

1. Add a capacity check and warning to the scheduler, together with breaks, spacing and peak window placement verified by a separate constraint checker;
2. Train on real, multi subject LMS data using a grouped split and a cross validated hyperparameter search;
3. Collect real confidence ratings in order to calibrate the label thresholds;
4. Validate the gain model and τ empirically, and personalise τ for each student;
5. Allow the system to recommend a total budget as well as a split; and
6. Carry out a formal usability study.

11. Ethical and Social Considerations

1. Telling a student "you are heading for a 44": The system's core output is a prediction about a person's future performance, delivered to that person at a stressful time. Such a prediction can become self fulfilling: a student told they are failing may disengage rather than work harder. The interface therefore presents the predicted mark as a current trajectory that the plan is designed to change, not as a verdict, and every prediction is accompanied by a concrete action (hours and sessions). This framing is a design decision, not a disclaimer.

2. The overconfidence label is a judgement about a person: Telling someone their self assessment is wrong is a stronger claim than giving them a number. The model's MAE is about 8 marks and the label uses a ±20 mark threshold, wider than the typical error, so the system does not call anyone miscalibrated on the basis of a small difference. The explanation shows its evidence ("you rated … but your predicted score is around …"), allowing the student to check the reasoning. Even so, a model of this accuracy should advise rather than pass judgement.

3. Synthetic data: The confidence column is simulated, so every conclusion about miscalibration rates describes our generator rather than real students. This is stated in the abstract, in the methods and here, not hidden in a footnote.

4. Bias and representativeness: Education_numerical.csv contains no demographic attributes, so we cannot measure whether predictions are better calibrated for some groups than for others. The absence of protected attributes does not mean the absence of bias: proxies such as class_size, resource_index and faculty_ratio reflect institutional resources and could encode socio economic differences. Any deployment would require a fairness audit on representative data.

5. Out of distribution predictions: As described in Section 10, subjects the model has never seen receive extrapolated predictions. In a real deployment, presenting these as though they were as reliable as in distribution predictions would mislead students, so the system should flag low confidence predictions explicitly.

6. Advisory, not authoritative: The system has no way of knowing that a student is ill, works night shifts or is a carer. The timetable is a suggestion only: the interface lets the student change any input and rerun the pipeline, and it does not treat deviation from the plan as failure.

7. Privacy and data protection: Study logs, attendance, quiz results and marks are personal data. The prototype runs locally, stores nothing beyond the trained model, and the LMS sync is simulated. A real deployment would need a lawful basis and consent under data protection law (for example the GDPR or Sri Lanka's Personal Data Protection Act No. 9 of 2022), together with data minimisation, a defined retention period, encryption and role based

access. Ethical frameworks for learning analytics (Slade & Prinsloo, 2013) also emphasise student agency and transparency about how data is used.

8. Staff use and power relations: The proposal identifies academic staff as secondary users. A risk flag seen by a tutor creates a record and a power imbalance. Appropriate safeguards would include: the student sees everything a tutor sees; flags are framed as the starting point of a conversation about a specific, correctable issue; and flags are never used in assessment or progression decisions.

9. Over reliance and autonomy: Automated planning could weaken students' own metacognitive skills. Showing why (the explanation) alongside what (the hours) is intended to build the student's calibration over time rather than replace it.

10. Academic integrity and disclosure of AI assistance: The team used AI assistants during development for code suggestions, debugging, generating test cases and editing report text. Every AI suggested component was reviewed, run and tested by the responsible member, and design decisions and their justifications are recorded in the decisions logs (docs/decisions_member1–3.md). All numerical results in this report come from reproducible runs of the project code (seed 42 unless stated otherwise); none were produced by an AI assistant.

12. Conclusion

This project designed, built and evaluated a four stage hybrid AI system that transforms a student's learning data and self reported confidence into a predicted mark, a calibration diagnosis with an explanation, an optimised division of study hours and a weekly timetable. Each stage uses the technique best suited to its kind of uncertainty, a neural network for statistical estimate, Mamdani fuzzy inference for a vague self report, genetic algorithm for combinatorial allocation, and rule based expert system for known procedural constraints.

Measured against the targets set at the proposal stage;

- The MLP achieves an MAE of 8.18 (target < 10) and outperforms the student average baseline (10.71). We showed openly that it is tied with linear regression and explained why, using correlation analysis, permutation importance and error analysis;
- the fuzzy system passes 12 of 12 acceptance tests, including continuity and monotonicity, and a hand worked example confirms its output (6.789);
- the GA beats the equal and proportional splits on 30 of 30 seeds with negligible variance, matches brute force enumeration, does not depend on its heuristic seeding, and runs in about a third of a second;
- the scheduler never violates its implemented constraints across 500 randomised cases, but it drops hours when the budget exceeds capacity and does not yet implement breaks or spacing; and
- An error propagation experiment showed that a 10 mark prediction error moves at most about 7 % of the study budget, so the pipeline degrades gradually.

The principal limitations are a dataset with weak signal, simulated confidence ratings, subjects outside the training data, a fitness model that has not been validated empirically, and an incomplete scheduler. Each of these has been documented together with its consequences and a plan for addressing it. The project's main contribution is a modular, testable and explainable pipeline in which every parameter is justified and every claim is supported by a reproducible number.

14. References

[1] N. J. Cepeda, H. Pashler, E. Vul, J. T. Wixted, and D. Rohrer, “Distributed practice in verbal recall tasks: A review and quantitative synthesis,” *Psychological Bulletin*, vol. 132, no. 3, pp. 354–380, 2006.

[2] C. A. Coello Coello, “Theoretical and numerical constraint handling techniques used with evolutionary algorithms: A survey of the state of the art,” *Computer Methods in Applied Mechanics and Engineering*, vol. 191, nos. 11–12, pp. 1245–1287, 2002.

[3] J. Dunlosky, K. A. Rawson, E. J. Marsh, M. J. Nathan, and D. T. Willingham, “Improving students' learning with effective learning techniques,” *Psychological Science in the Public Interest*, vol. 14, no. 1, pp. 4–58, 2013.

[4] L. J. Eshelman and J. D. Schaffer, “Real coded genetic algorithms and interval schemata,” in *Foundations of Genetic Algorithms 2*, pp. 187–202. San Mateo, CA, USA: Morgan Kaufmann, 1993.

[5] F. A. Fortin, F. M. De Rainville, M. A. Gardner, M. Parizeau, and C. Gagné, “DEAP: Evolutionary algorithms made easy,” *Journal of Machine Learning Research*, vol. 13, pp. 2171–2175, 2012.

[6] J. C. Giarratano and G. D. Riley, *Expert Systems: Principles and Programming*, 4th ed. Boston, MA, USA: Thomson Course Technology, 2005.

[7] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *Proc. 13th Int. Conf. Artif. Intell. Statist. (AISTATS)*, pp. 249–256, 2010.

[8] D. E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*. Reading, MA, USA: Addison Wesley, 1989.

[9] Group 4, *Implementation Handbook – Smart Learning and Performance Optimization System*, ver. 1.0, Internal Project Document, 2026.

[10] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI, USA: University of Michigan Press, 1975.

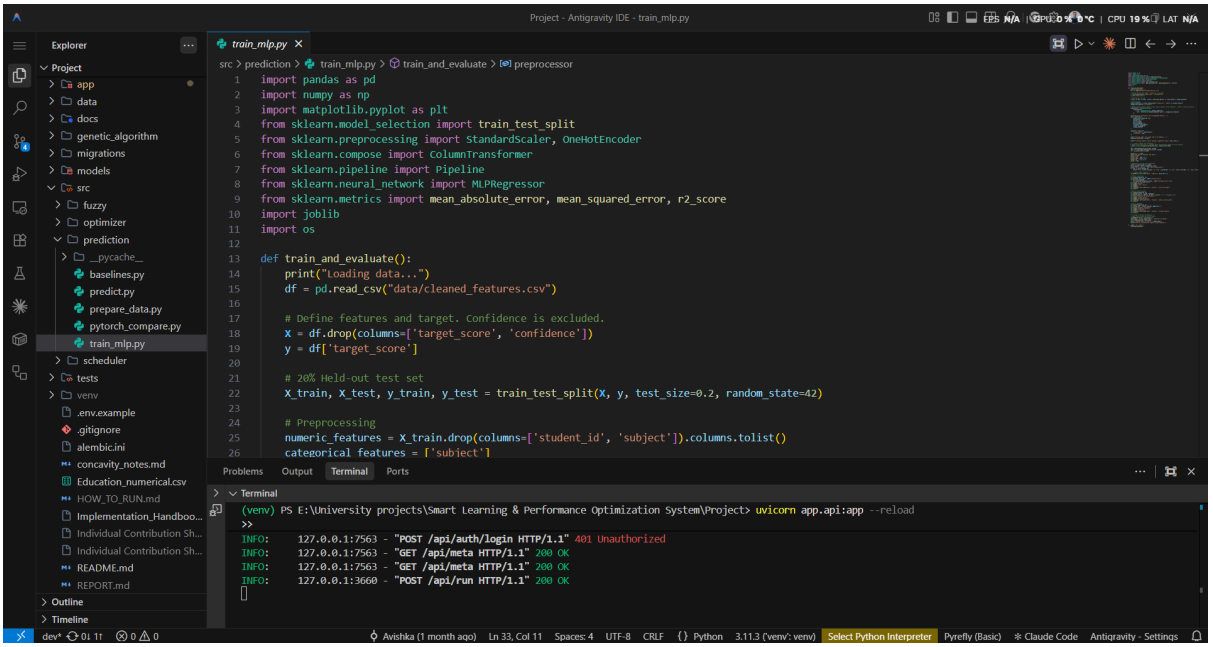
15. Individual contribution Sheets

Student Name	EAA Hashara	**Registration No.**	D/BCS/25/0016
Degree / Intake	Intake 42, CS	**Group No.**	54
Project Title	Smart Learning and Performance Optimization System		

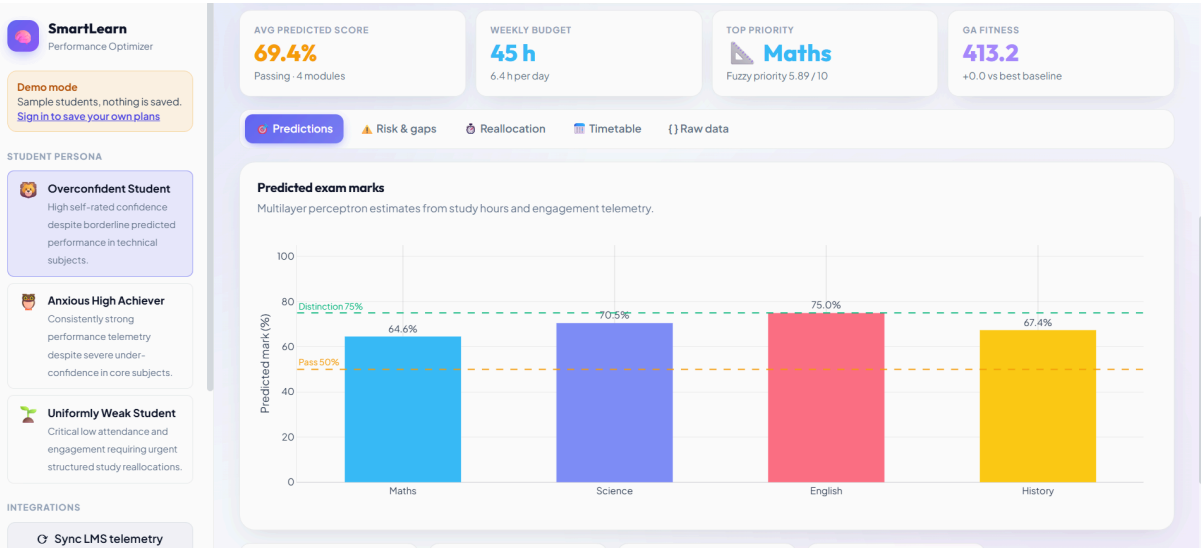
Technical Contributions:

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
Data preparation & leakage audit	Audited dataset and removed 4 target-leaking columns Reshaped 10,000 students into 40,000 student–subject rows Cleaned features and synthesised confidence column Wrote data dictionary	Python, pandas, NumPy Data preprocessing / feature engineering	src/prediction/prepare_data.py docs/data_dictionary.md
Stage 1 – Neural-network prediction engine	Designed and trained MLP regressor (64→32 ReLU, Adam, early stopping) predicting mark per subject Test MAE 8.18 (target < 10) predict() wrapper clips output 0–100 for Stage 2	scikit-learn MLPRegressor, OneHotEncoder Supervised learning – Artificial Neural Network (MLP)	src/prediction/train_mlp.py, predict.py Saved model (models/) loss_curve.png, pred_vs_actual.png, residuals.png
Model evaluation & baselines	Built global-mean, student-average and linear-regression baselines (MLP beats student-average MAE 10.71) Permutation importance and error analysis PyTorch version of the MLP for comparison	scikit-learn, PyTorch, matplotlib Regression metrics (MAE, RMSE, R^2)	src/prediction/baselines.py, pytorch_compare.py Report Section 9.1 docs/decisions_member1.md

System integration (4-stage hybrid pipeline)	Defined frozen interfaces between all stages Integrated NN → fuzzy → GA → scheduler into one pipeline End-to-end smoke test with 3	Python, NumPy Hybrid AI: ANN + Mamdani fuzzy logic + genetic algorithm + rule-based system	smoke_test.py Integration commits on GitHub (branch dev) Report Sections 7 & 9
Backend, user interface & persistence	FastAPI REST backend with user accounts and authentication SQLite database with Alembic migrations Web front-end and Streamlit dashboard; API tests Final report and presentation script	FastAPI, SQLAlchemy, Alembic, Streamlit, HTML/CSS/JS, pytest	app/ (api.py, auth.py, study.py, web/) tests/test_api.py REPORT.md, docs/presend.md



Code : MLPRegressor(....)



UI : Predicted Marks

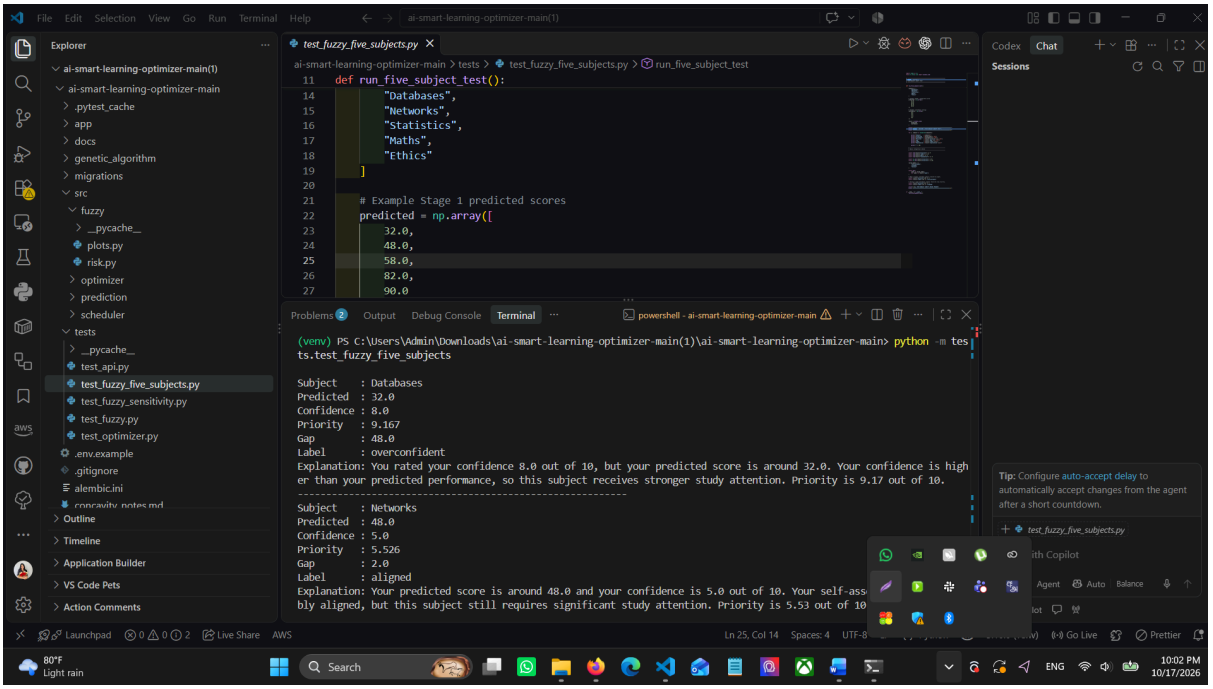
I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature: _____ **Date:** 16/10/2026

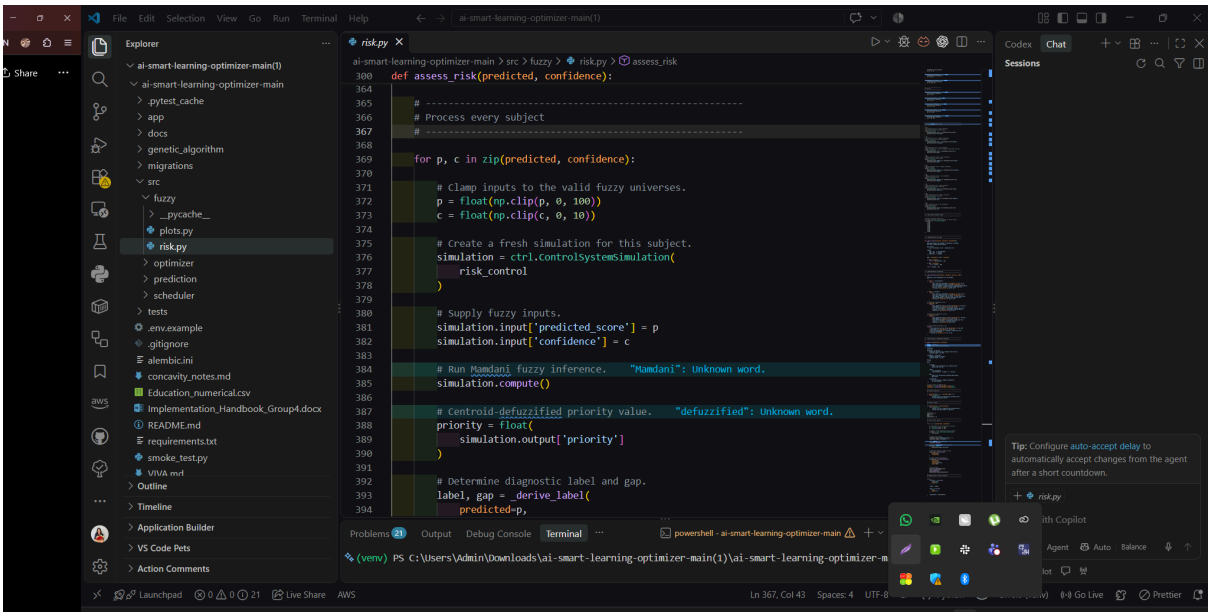
Student Name	M Sadiyan	**Registration No.**	D/BIT/24/0009
Degree / Intake	Intake 41, BIT	**Group No.**	54
Project Title	Smart Learning and Performance Optimization System		

Technical Contributions:

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
Fuzzy Membership Function Design	Designed fuzzy membership functions for predicted marks, student confidence, and study priority.	Python, scikit-fuzzy, triangular and trapezoidal membership functions	Membership function code and graphs
Mamdani Fuzzy Inference System	Implemented a Mamdani fuzzy inference system to assess student learning risks and study priorities.	Mamdani Fuzzy Inference, Python, scikit-fuzzy	Fuzzy inference code and assessment outputs
Fuzzy Rule Base Development	Developed a nine-rule fuzzy rule base to evaluate different student performance and confidence conditions.	Fuzzy IF–THEN rules, Mamdani inference	Nine-rule implementation and rule evaluation results
Risk Score Calculation	Applied centroid defuzzification to generate numerical study priority scores ranging from 0 to 10.	Centroid defuzzification, fuzzy reasoning	Calculated risk scores and sample outputs
Gap Classification and Explanation	Developed confidence-gap classification and explanation logic to identify overconfident, aligned, and anxious students.	Python, confidence-gap analysis, rule-based explanations	Classification results and generated explanations



Fuzzy Inference System Testing and Validation



Implementation of the Fuzzy Risk Assessment Function

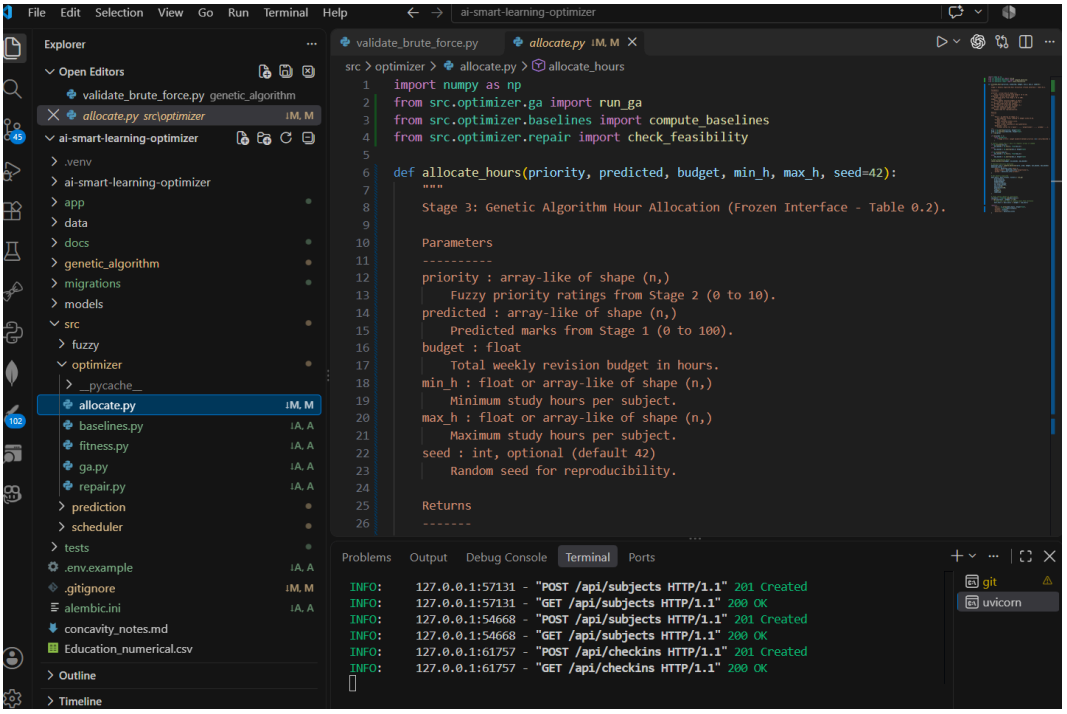
I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature: _____ **Date:** 16/10/2026

Student Name	HEJM Perera	**Registration No.**	D/BIT/24/0075
Degree / Intake	Intake 41, BIT	**Group No.**	54
Project Title	Smart Learning and Performance Optimization System		

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
Genetic Algorithm Setup	Configured the DEAP framework and developed the genetic optimization loop.	Python, DEAP; optimizer implementation	src/optimizer/allocate.py - Genetic Algorithm implementation and optimization loop.
Fitness Function	Implemented a diminishing returns fitness function incorporating headroom and penalty terms.	Genetic Algorithm; fitness-function code	Source code implementing diminishing-returns fitness, headroom and penalty terms.
Constraint Handling	Developed a bounded repair operator to keep allocations within specified limits.	Python; repair operator and allocation validation	Bounded repair operator code and allocation outputs demonstrating constraint handling.
Baseline Algorithms	Implemented equal, proportional and greedy allocation methods for comparison.	Baseline algorithms; comparison results	Implementation and comparison results for equal, proportional and greedy allocation methods.
Optimization Verification	Conducted concavity verification to assess the optimization behaviour.	Mathematical verification; concavity analysis	Concavity verification results and supporting analysis.
Pipeline Integration	Integrated the optimizer into the core project pipeline.	src/optimizer/allocate.py;integration code	src/optimizer/allocate.py and execution output showing the optimizer integrated into the core pipeline.

1. Genetic Algorithm Setup



2. Genetic Algorithm Output – Weekly Study Allocation

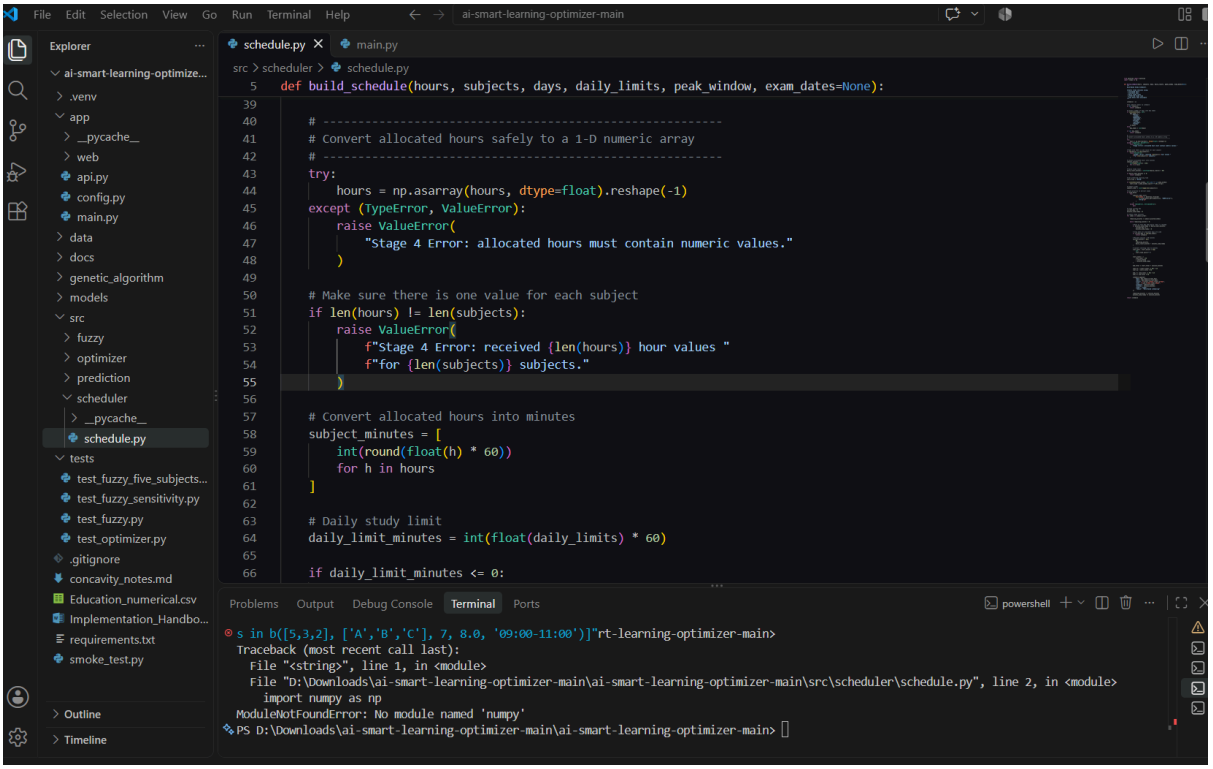


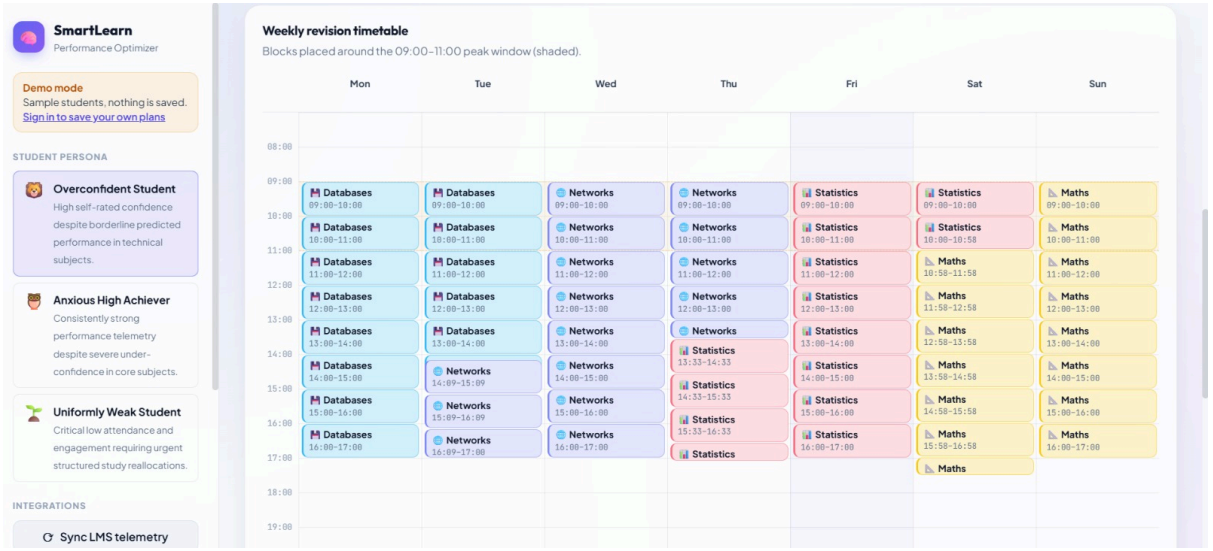
I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature:  **Date:** 16/10/2026

Student Name	KSPS Amarasinghe	**Registration No.**	D/BSE/25/0023
Degree / Intake	Intake 42,SE	**Group No.**	54
Project Title	Smart Learning and Performance Optimization System		

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
Rule-Based Scheduler	Implemented the scheduler to convert allocated study hours into a weekly timetable.	Python, rule-based scheduling; src/scheduler/schedule.py	src/scheduler/schedule.py - Python implementation of the build_schedule() function, which converts allocated study hours into timetable sessions containing the day, start time, end time, subject, duration, and reason.
Study-Hour Constraints	Implemented daily study limits to ensure generated schedules follow the defined constraints.	Rule-based constraints; scheduling logic	daily limit logic in schedule.py; timetable days end at 17:00 (8-hour limit)
Study-Block Generation	Developed functionality to generate scheduled study blocks.	Python; generated study blocks	Sessions of at most 60 minutes (min(60, …) in schedule.py); blocks shown in the Weekly revision timetable of the SmartLearn app
Weekly Timetable	Implemented weekly timetable generation based on allocated study hours.	Rule-Based Scheduler; generated weekly timetable	Monday–Sunday screenshot of the Weekly revision timetable in the SmartLearn app
Schedule Validation	Manually checked the generated timetable against the defined scheduling limits.	Scheduler execution; manual verification of the output	Weekly timetable from a run with the Overconfident Student persona: no session over 60 minutes and no day over 8 hours (09:00–17:00); 56 hours scheduled, which is the full 7-day × 8-hour capacity





I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature:  **Date:** 16/10/2026